

5. CNNs & Object Detection

CS 4243 Computer Vision & Pattern Recognition

Angela Yao

Recap & Outline

Last Week

- Gabor Filters
- Filter banks
- Multiscale reasoning
- Textures
- Precursor to CNNs

Today's Lecture

- Image Classification
- Neural Networks & MLPs
- Convolutional Neural Networks
- Object Detection
- CNN-Based Object Detection

Image Classification: a Core Task in CV

mathematical function representing the classifier

$$\begin{aligned} f(\text{apple image}) &= \text{"apple"} \\ f(\text{tomato image}) &= \text{"tomato"} \\ f(\text{cow image}) &= \text{"cow"} \end{aligned}$$

input image output label

$$\mathbf{x} \in \mathbb{R}^{H \times W \times C}$$
$$f_{\theta}(\mathbf{x}) = \mathbf{z} \in \mathbb{R}^K$$

$\mathbf{x}$: input image
 H, W, C : height, width, # of channels
 f_{θ} : classifier with parameters θ

- Assign discrete label of image from a given set of K classes
e.g. {cat, dog, cow, apple, tomato, ...}.

$$\hat{y} = \arg \max_{k \in \{1, \dots, K\}} z_k$$

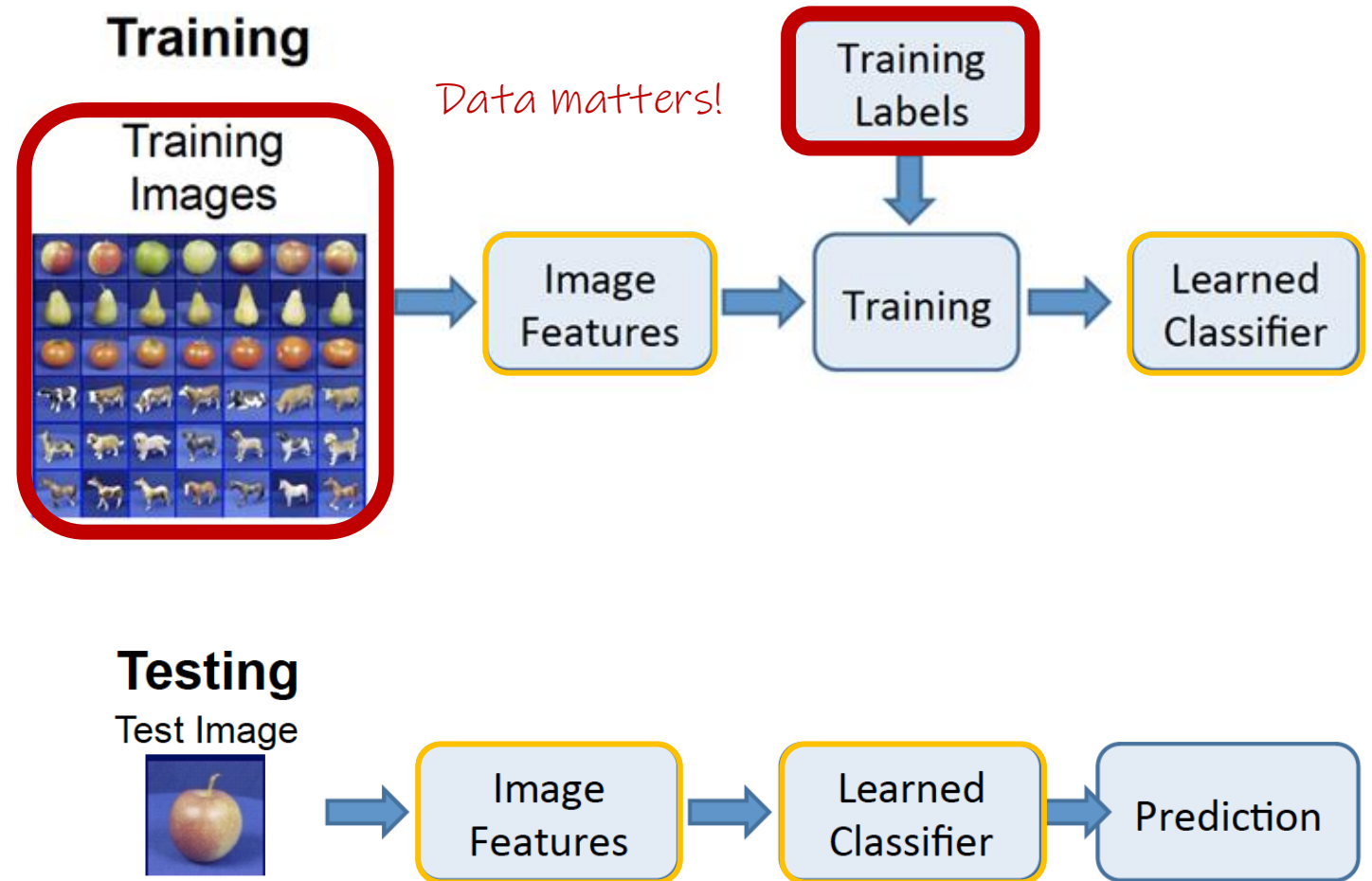
- Evaluation is based on accuracy
(# of samples correct / total samples):

$$\text{Accuracy} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[\hat{y}_i = y_i].$$

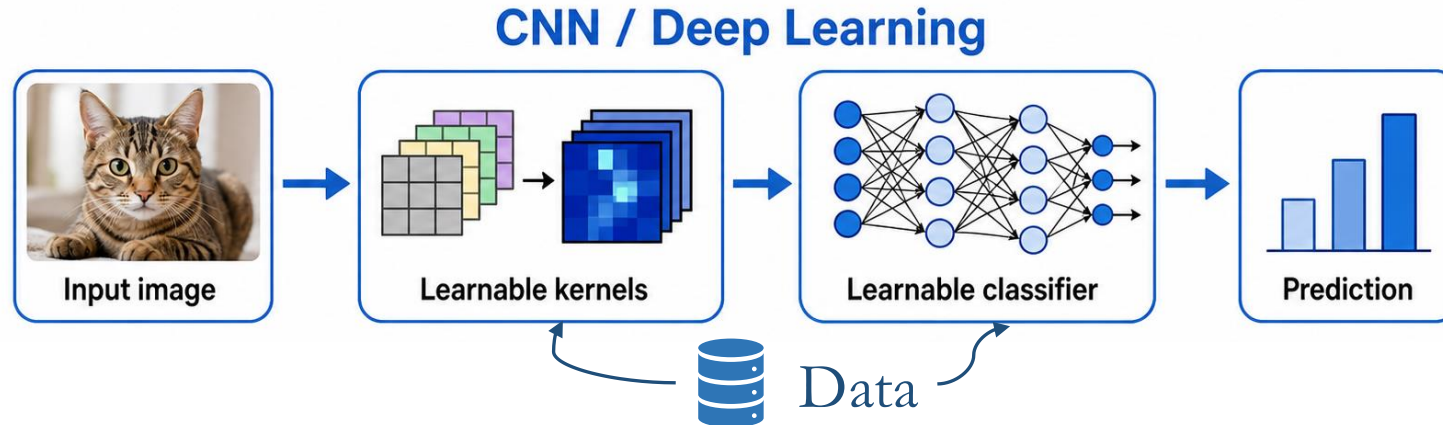
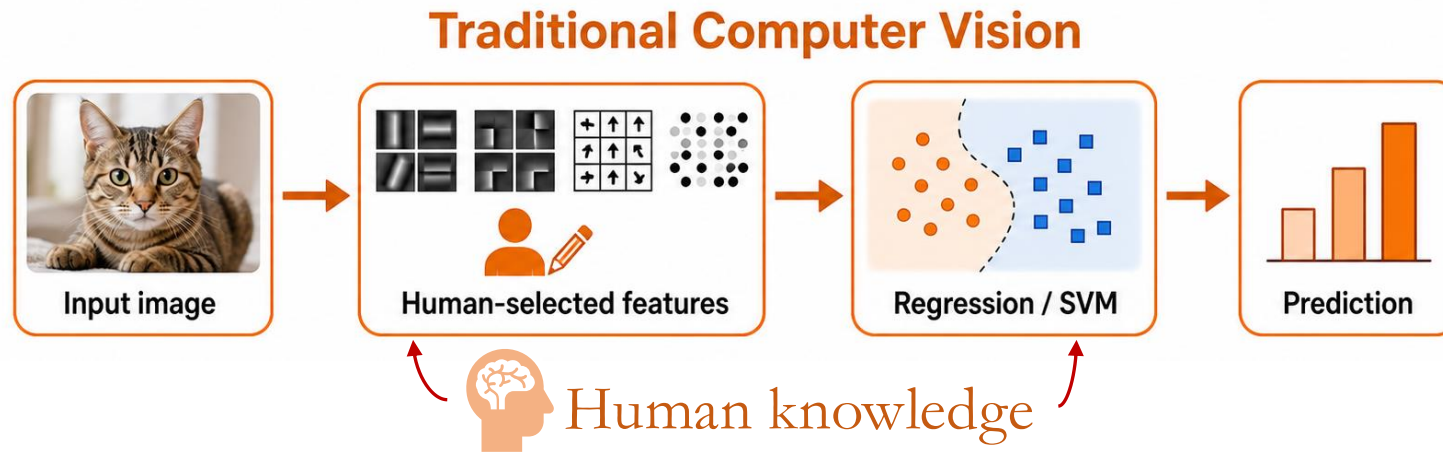
i : sample index
 N : total samples
 y_i : ground truth label of sample i

Data-Driven Image Classification

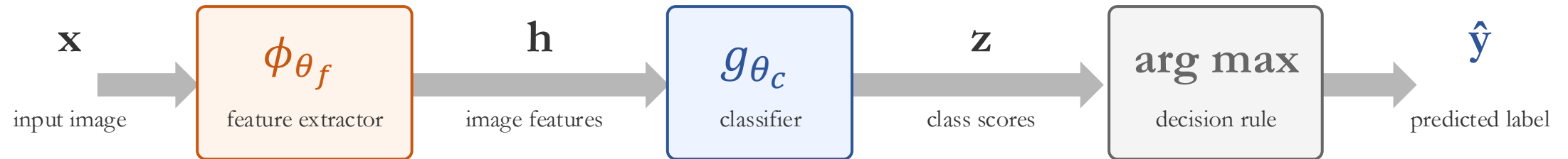
- Collect a database of images with labels
- Use machine learning to train a classifier based on labelled training samples
- Tune classifier hyper-parameters on validation set
- Evaluate classifier on unseen test set of images



Learning Replaces Hand-Selected Components



Stages of A Classifier



$$\mathbf{h} = \phi_{\theta_f}(\mathbf{x}) \in \mathbb{R}^d$$

ϕ maps pixels to a feature representation

$$\mathbf{z} = g_{\theta_c}(\mathbf{h}) \in \mathbb{R}^K$$

g maps features to one score per class

$$\hat{y} = \arg \max_k z_k$$

select the class with the largest score

Complete prediction function:
$$\hat{y} = f_{\theta}(\mathbf{x}) = \arg \max_k [g_{\theta_c}(\phi_{\theta_f}(\mathbf{x}))]_k$$

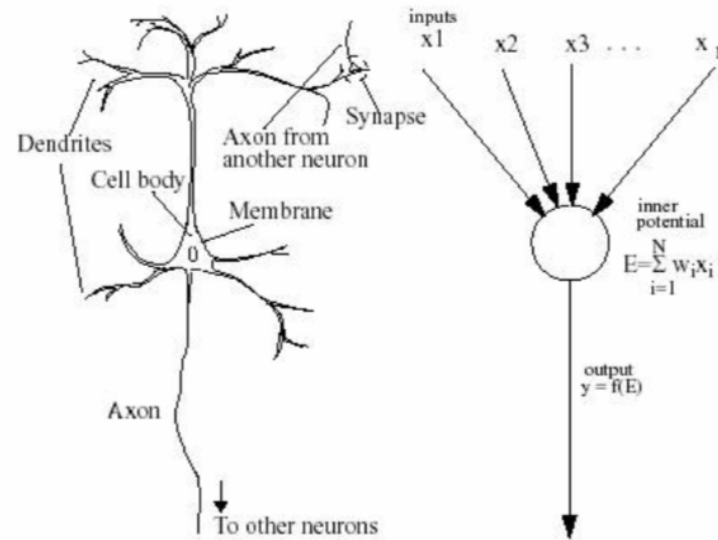
- θ_f is fixed and human-designed for traditional CV and learned for deep learning
- θ_c is learned for both traditional CV and deep learning
- $[\theta_f, \theta_c]$ are learned together (“end-to-end”) for deep learning

Neural Networks

Perceptrons, Multi-Layer Perceptrons

“Neural” Networks?

The origins of neural networks come from efforts of mathematically representing information processing in biological systems (hence the term “neural”).

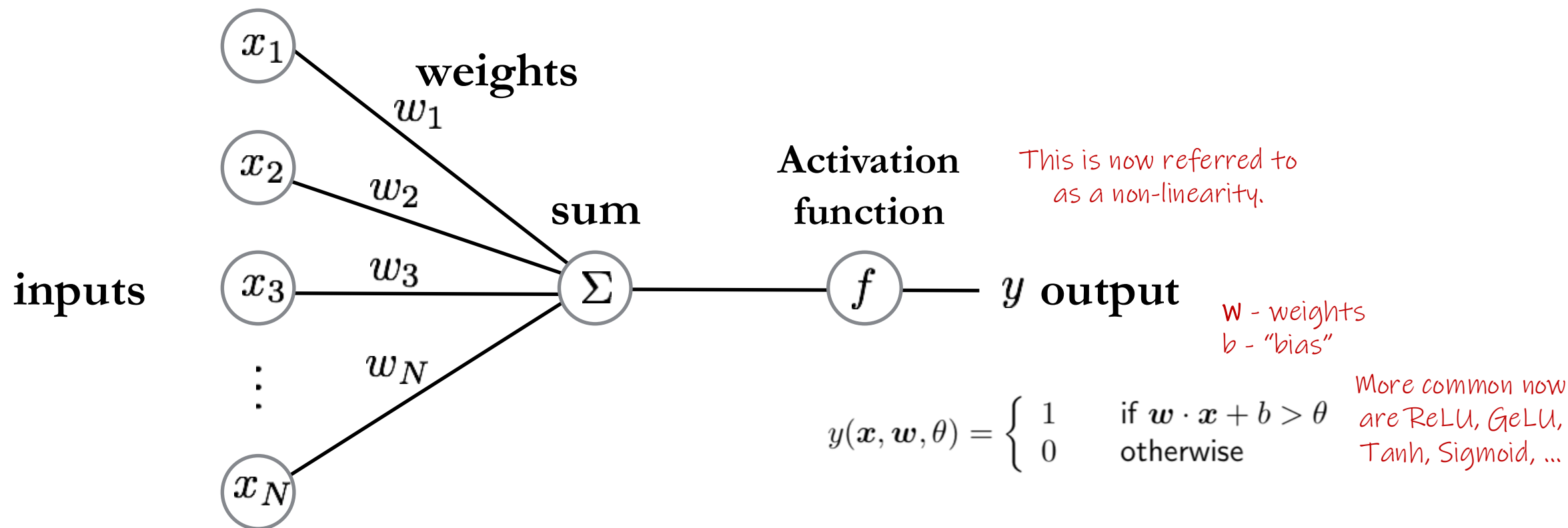


Neural nets/perceptrons are **loosely** inspired by biology. But they certainly are **not** a model of how the brain works, or even how neurons work.

<http://commonsenseatheism.com/wp-content/uploads/2011/12/biological-and-artificial-neurons.jpg>

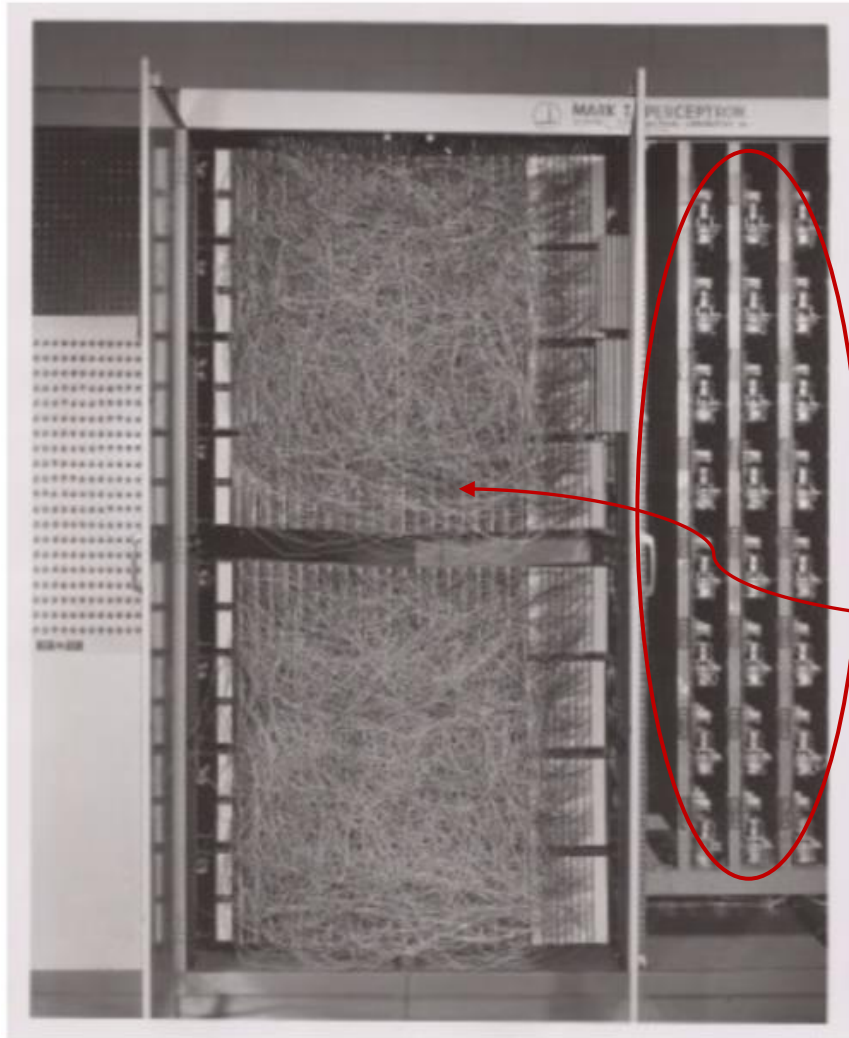
Our interests in neural networks are less about biological systems and more about efficient models for statistical pattern recognition.

The Perceptron (a “neuron”)



The Perceptron “algorithm” (really a machine) was invented in 1957 by Frank Rosenblatt to perform image recognition. It consisted of a single layer (*i.e.* no hidden units) with a step function to threshold the output:

The Perceptron Machine (1957)



The Mark I Perceptron machine was a hardware implementation of the perceptron algorithm.

The Perceptron machine was connected to a 20x20 array of photocells producing a 400 pixel image (essentially a camera).

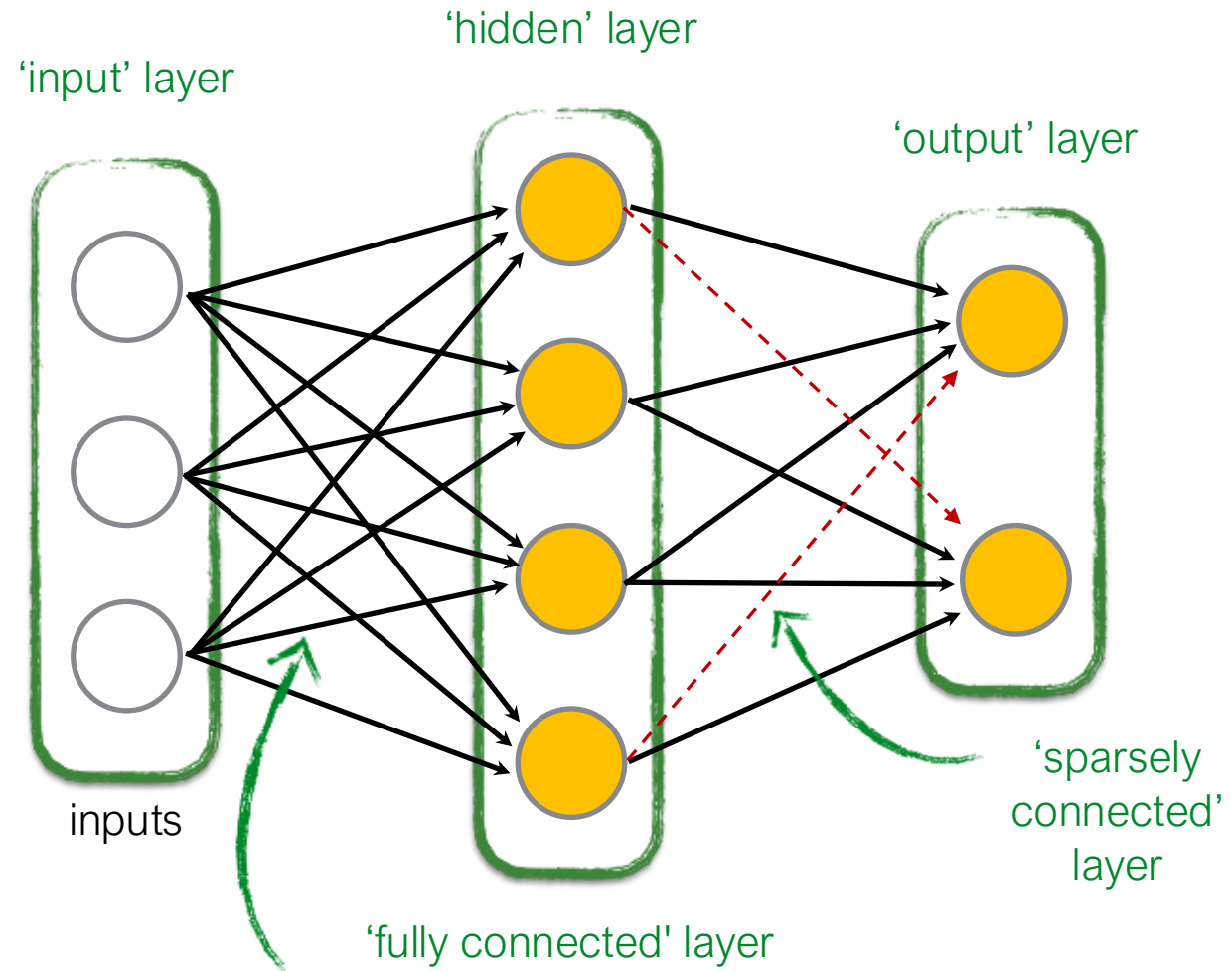
The nest of wires create different combinations of inputs.

Arrays of potentiometers (adjustable variable resistors) which could be used as adaptive ("hand-tuned") weights.

A Neural Network is Simply ..

... connecting a bunch of perceptrons together!

Perceptron outputs can be used as inputs to further perceptrons, creating a multi-layer perceptron (MLP).



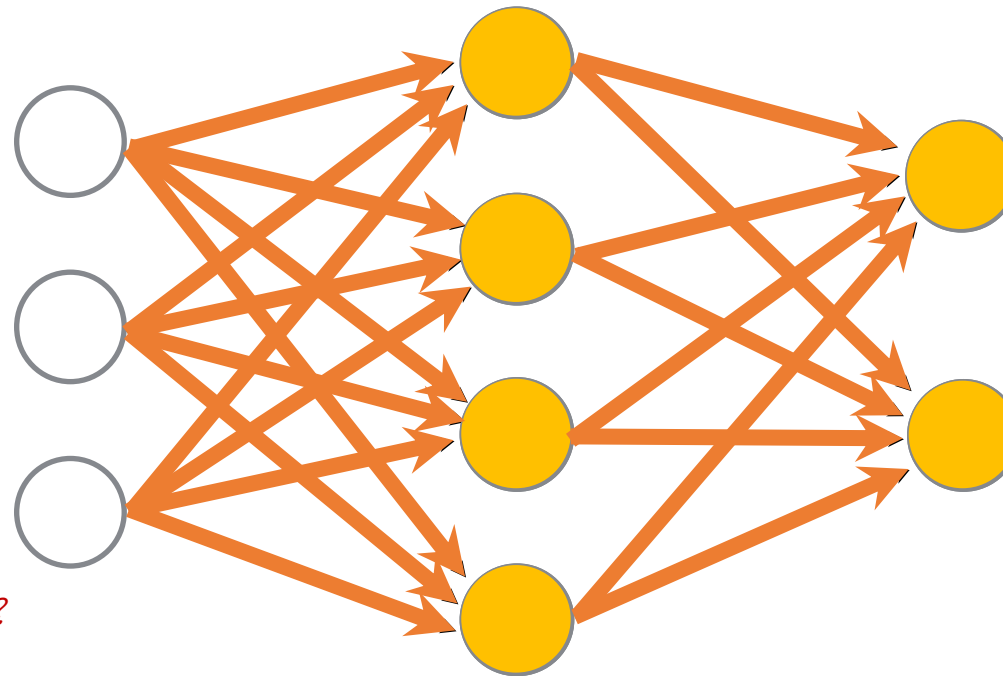
all pairwise neurons between layers are connected

How many neurons (perceptrons)?

$$4 + 2 = 6$$

How many weights (edges)?

$$(3 \times 4) + (4 \times 2) = 20$$



How to give image as input to a MLP?
Flatten matrix into a vector so that
each pixel is an input dimension.

Also referred to as Fully-Connected Network (FCN)

How many learnable parameters total?

$$20 + \underset{\text{bias terms}}{(4 + 2)} = 26$$

MLP Formalism

Input: $\mathbf{a}^{(0)} = \mathbf{u}$

$\mathbf{u}$: input feature vector

can be $\text{vec}(\mathbf{x})$ if it is the first layer for image,
 $\mathbf{h}$ if MLP is the classification head (next slide)

Hidden layers:

$$\mathbf{a}^{(\ell)} = \underbrace{\sigma(W^{(\ell)} \mathbf{a}^{(\ell-1)} + \mathbf{b}^{(\ell)})}_{\text{Linear transformation of previous layer } \ell-1 \text{'s outputs}}, \ell = 1, \dots, L-1$$

ReLU,
sigmoid, etc.

Linear transformation of
previous layer $\ell-1$'s outputs

$\mathbf{a}^{(\ell)}$: representation in layer ℓ .

$W^{(\ell)}$: weights in layer ℓ .

$\mathbf{b}^{(\ell)}$: biases in layer ℓ .

σ : non-linear activation function

Output** layer:

$$\mathbf{z} = \mathbf{a}^{(L)} = W^{(L)} \mathbf{a}^{(L-1)} + \mathbf{b}^{(L)}$$

** Ambiguity for the output layer of the MLP, depending on whether it is used as a general module within a network, if it is used as the classifier head, or if it is used as a feature and classifier. Within a network, the "output" is the same as a hidden layer. For an MLP that serves as a complete feature extractor and classifier, to be consistent with slide 6, $\mathbf{x} \xrightarrow{\phi_{\theta_f}} \mathbf{h} \xrightarrow{g_{\theta_c}} \mathbf{z} \xrightarrow{\arg \max} \hat{y}$, one interpretation is to see $\theta_f = \{W^{(\ell)}, \mathbf{b}^{(\ell)}\}_{\ell=1}^{L-1}$ and $\theta_c = \{W^{(L)}, \mathbf{b}^{(L)}\}$, where $\mathbf{h} = \mathbf{a}^{(L-1)} = \phi_{\theta_f}(\mathbf{u})$.

Are MLPs sufficient for Computer Vision?

- Applying an MLP to a raw image requires flattening the image into a vector

$$\mathbf{x} \in \mathbb{R}^{H \times W \times C} \longrightarrow \text{flatten}(\mathbf{x}) \in \mathbb{R}^{HWC}$$

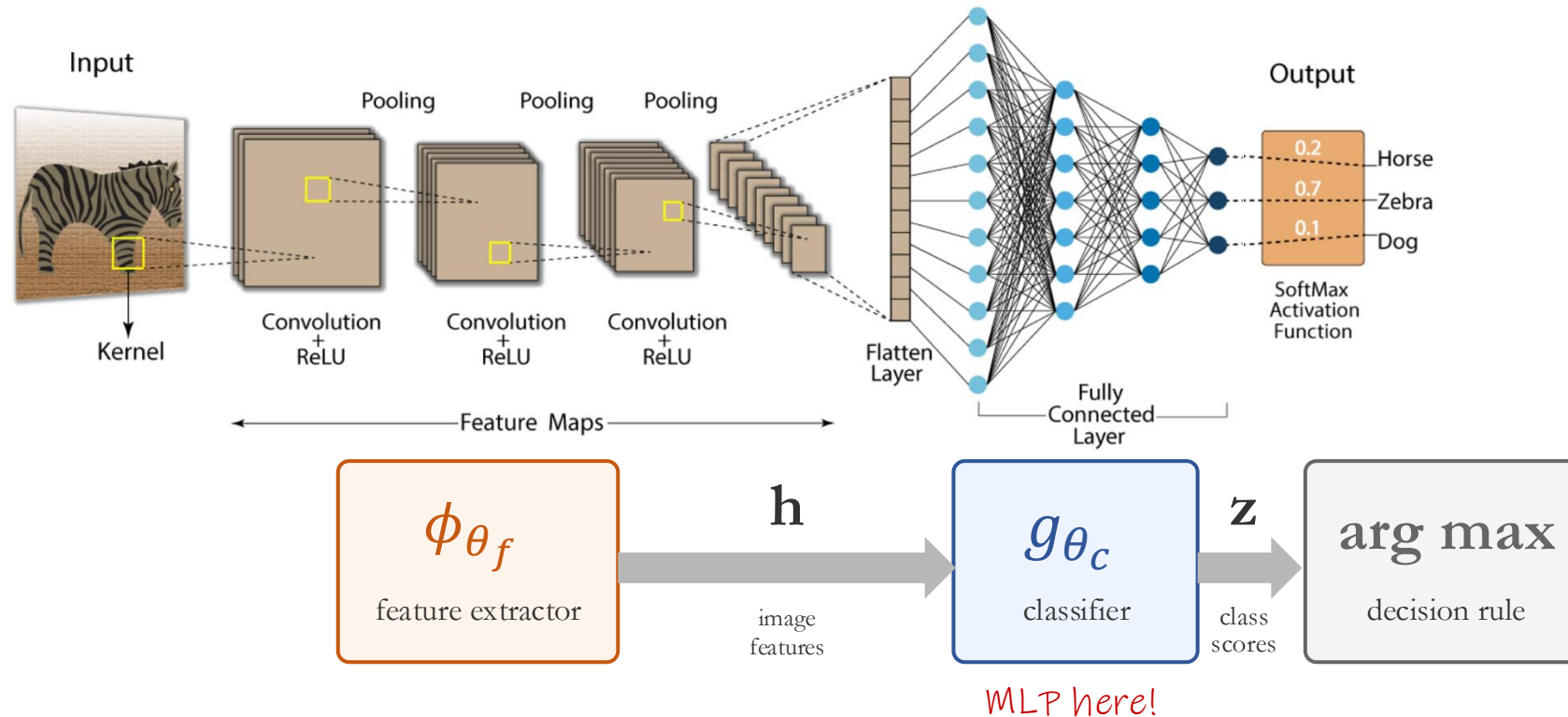
- Every hidden unit connects to every pixel
- Flattening destroys the inter-pixel relationship in spatial neighbourhoods
- The number of weights grows rapidly with image size. $(H * W * C)$

$$224 \times 224 \times 3 \xrightarrow{M=1000} 150,528,000 \text{ weights}$$

- Therefore, MLPs are used only selectively as internal modules

MLPs in Convolutional Neural Networks

- In CNNs, an MLP often forms the classification head, as part of the last fully connected layer(s).

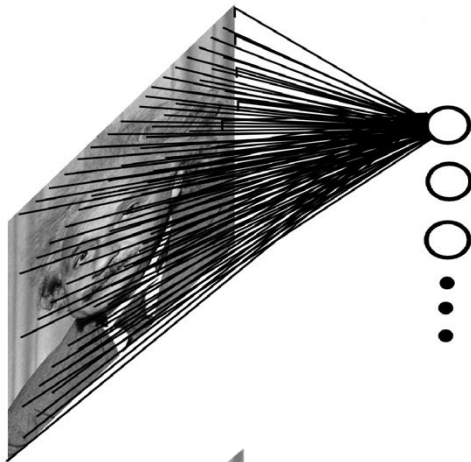


Convolutional Neural Networks

Convolution Layers vs. Perceptrons, General CNN backbones,
ConvNet Architectures, Loss functions

Images as Inputs to Neural Networks

Suppose we have a 200x200 image with 30k hidden units in 1 layer.



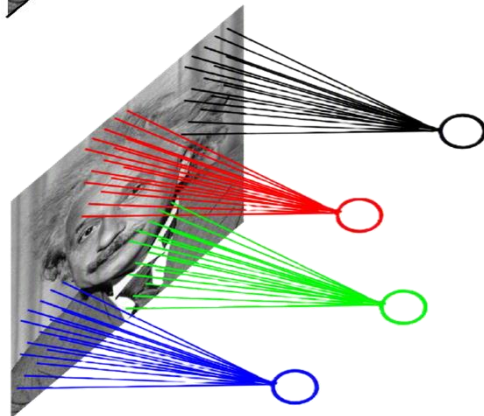
How many parameters for fully-connected network?

$$(200 \times 200) * 30k + 30k = \sim 1.2 \text{ billion params!}$$

weights

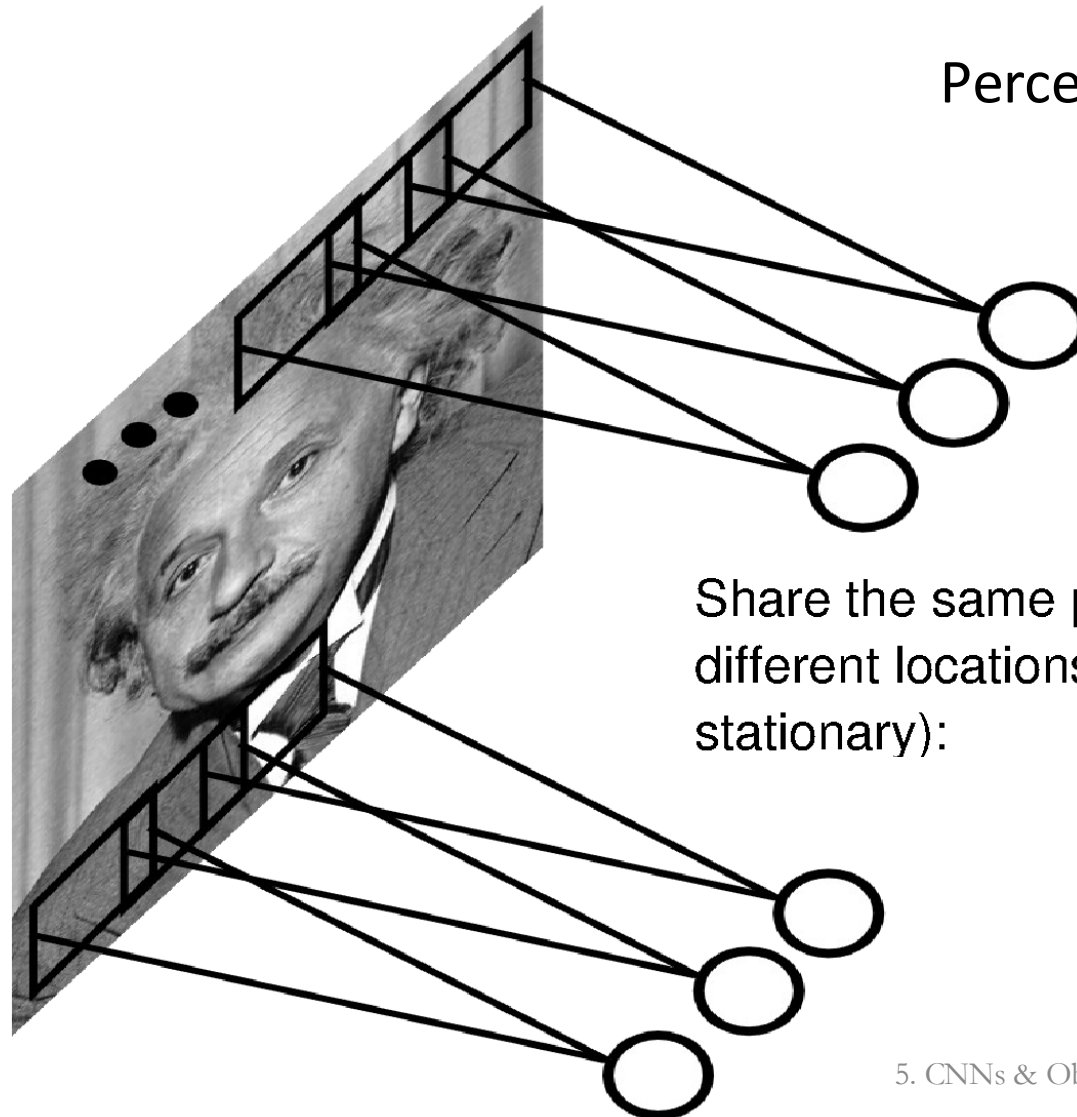
Biases (ignored for approximation)

Fully connected hidden units ignores a key property of images: pixels near to each other are more correlated than pixels far apart.



Each hidden unit contains 10x10 connection instead of entire image: $(10 \times 10) * 30k = \sim 3 \text{ million params!}$

Convolution Share Kernels Across Locations



Perceptron:
$$\text{output} = \begin{cases} 0 & \text{if } w \cdot x + b \leq 0 \\ 1 & \text{if } w \cdot x + b > 0 \end{cases}$$

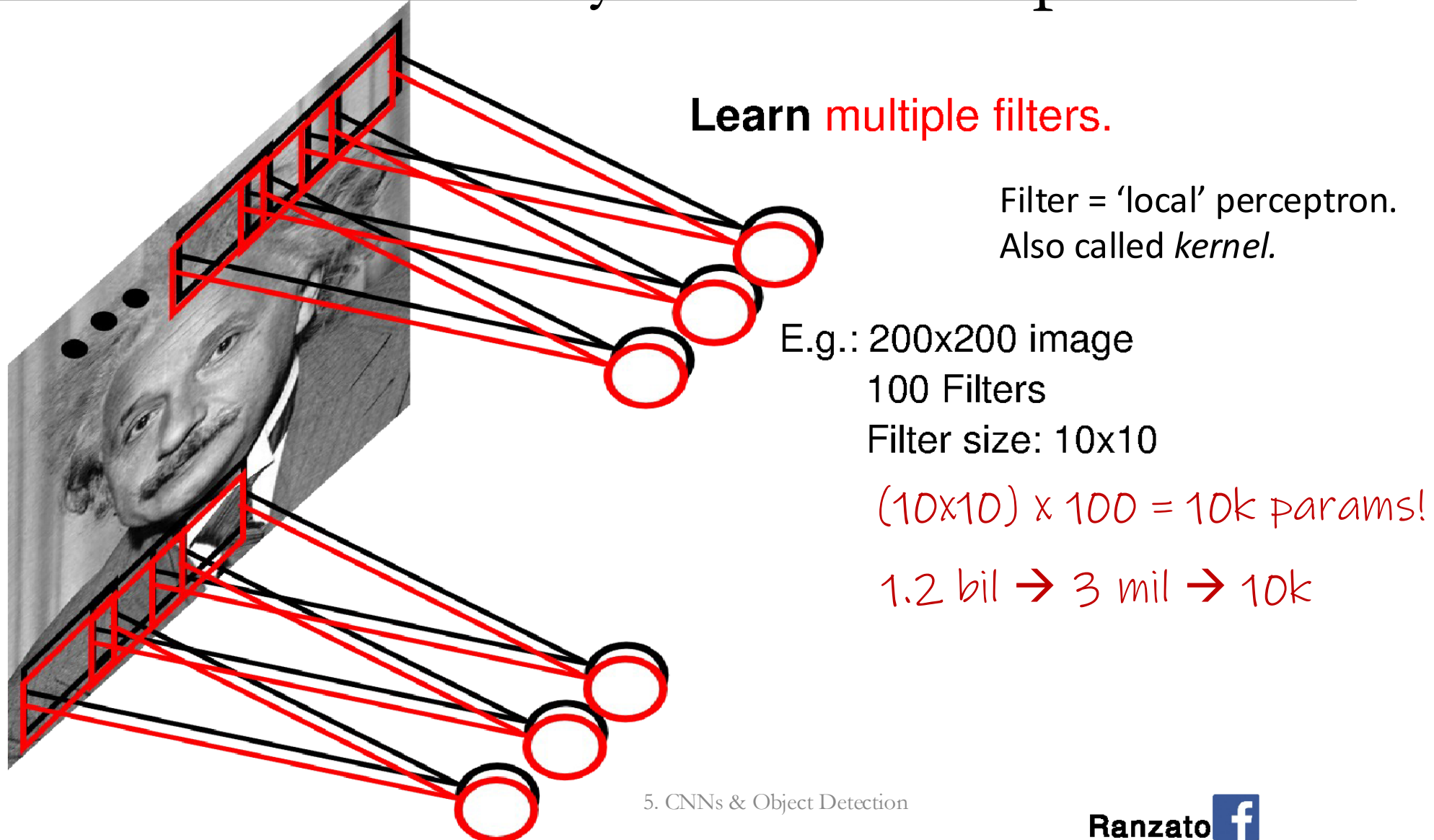
$$w \cdot x \equiv \sum_j w_j x_j$$

This is convolution!

We are doing convolution with a learned kernel. The kernel has only 100 parameters (10x10).

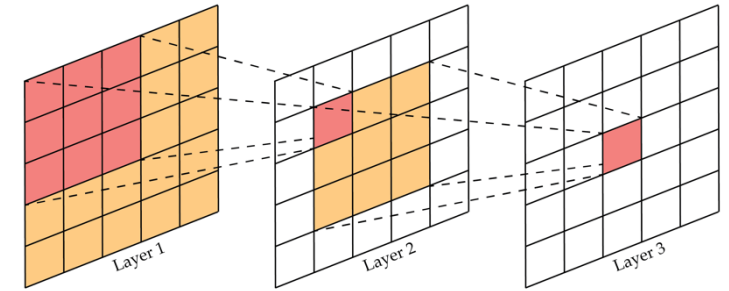
Share the same parameters across different locations (assuming input is stationary):

A Convolution Layer Has Multiple Filters



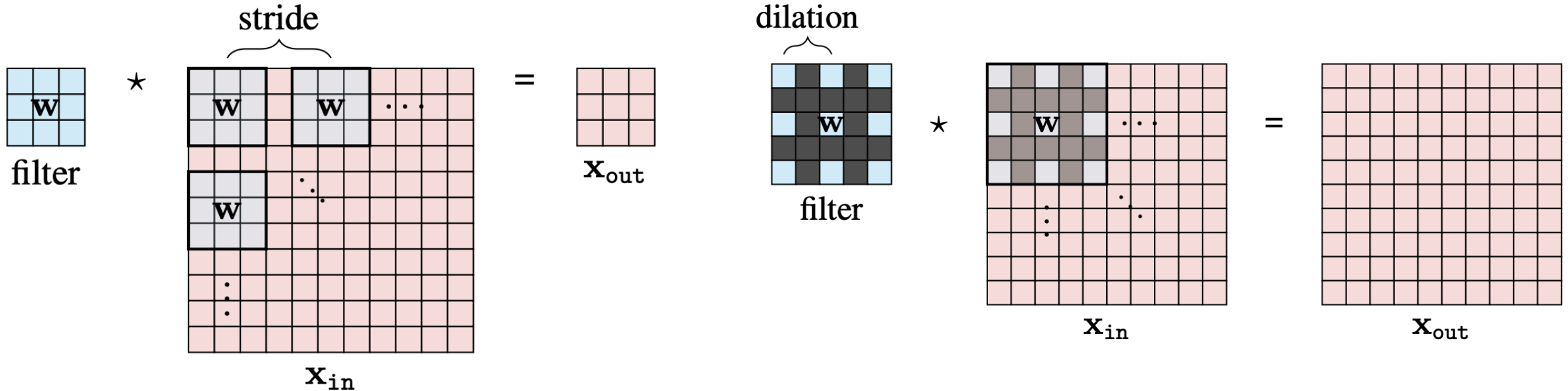
Why Convolution Suits Image Analysis

- Local interactions – *receptive fields*
 - Assume that in an image, we care about ‘local neighborhoods’ only for a given neural network layer.
 - Composition of layers will expand local -> global.
- Parameter sharing (built into convolution by design)
 - Stationarity assumption, use same weights for more than one perceptron in the neural network.
 - Leads to *equivariant representation*
 - If input changes (e.g., translates), then output changes similarly



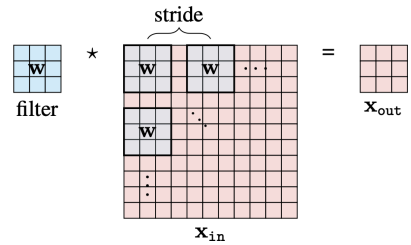
The receptive field of a neural unit or one output value in a feature map is the region whose values can influence that unit's activation. It is small locally, but expands wrt the original image's input.

Strided & Dilated Convolutions



- **Strided convolution:** Down-samples the feature map, reducing spatial resolution and computation while allowing deeper layers to represent increasingly larger image regions.
- **Dilated convolution:** Enlarges the receptive field without adding parameters or reducing spatial resolution—useful when dense, spatially precise predictions are required e.g. in segmentation.

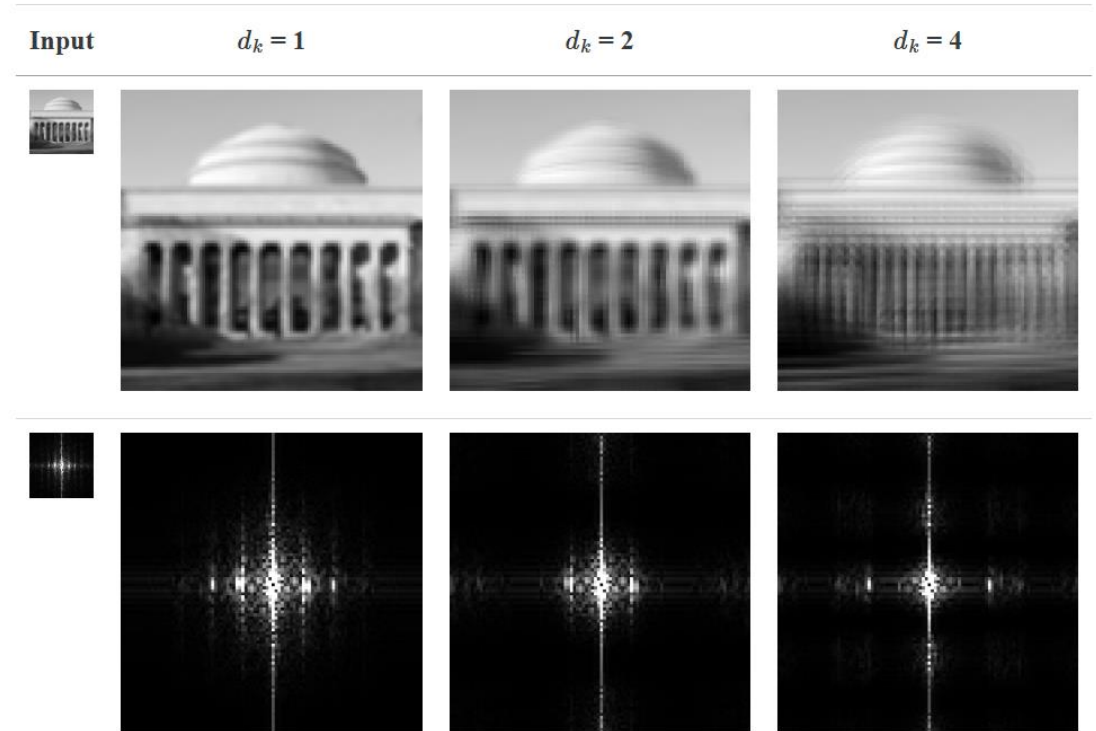
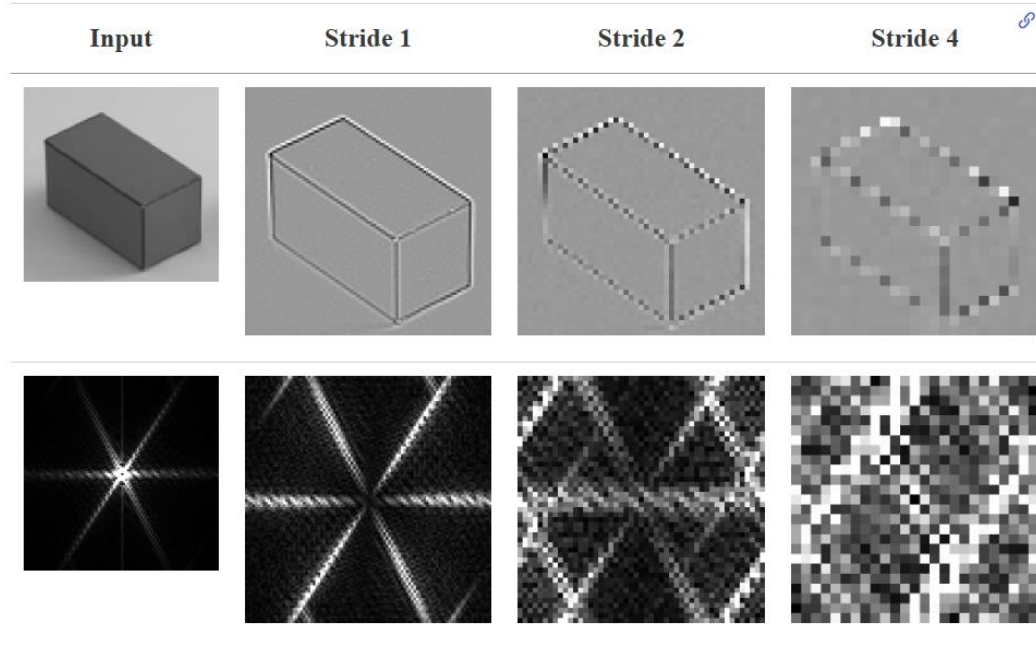
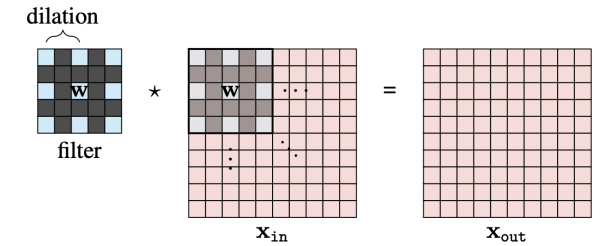
Strided Convolution, Dilated Convolution



$$H_{out} = \left\lfloor \frac{H + 2P - D(K-1) - 1}{S} \right\rfloor = \left\lfloor \frac{H + 2P - K}{S} \right\rfloor$$

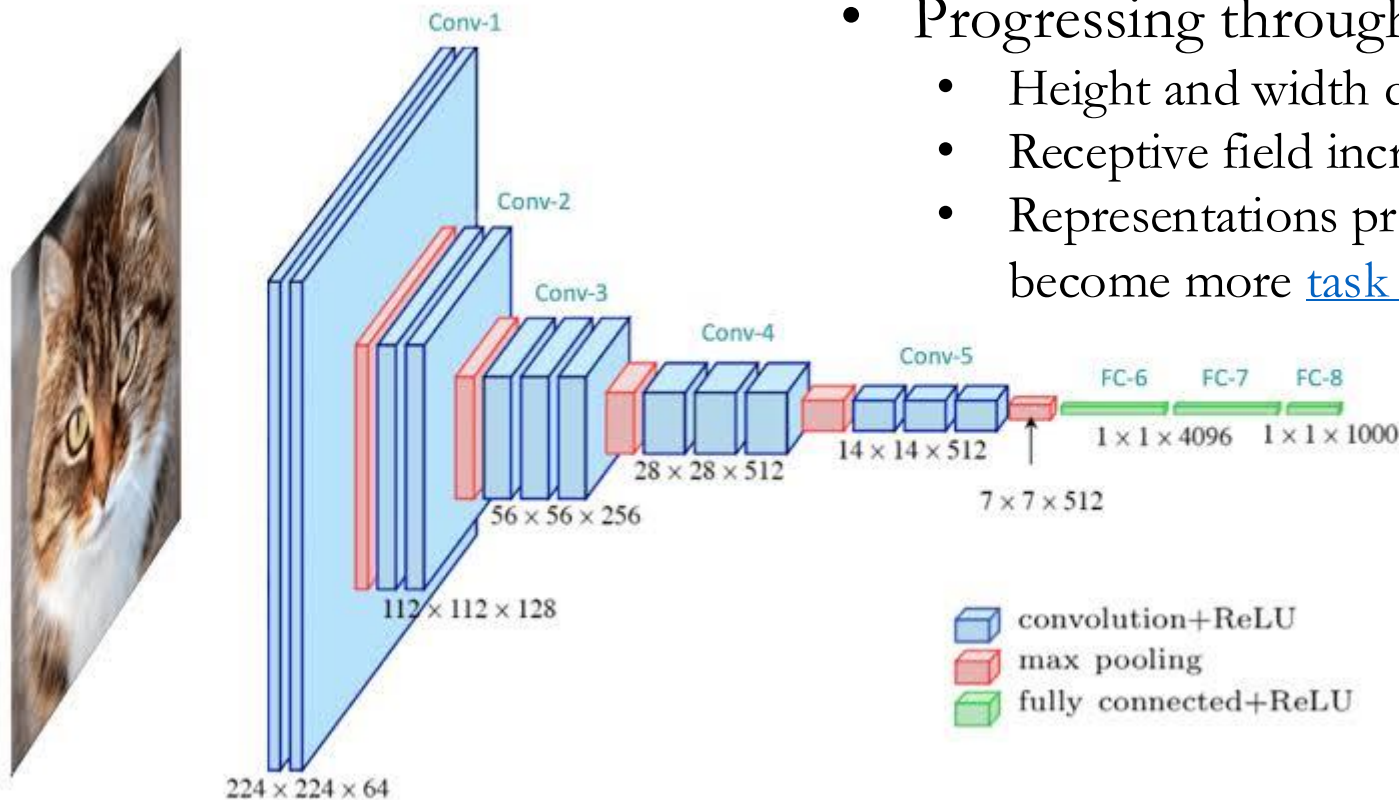
Annotations for the formula:

- H : input feature map size
- $2P$: padding
- D : dilation
- K : kernel size
- S : stride
- $\lfloor \cdot \rfloor$: floor operation
- If no dilation: $H_{out} = \left\lfloor \frac{H + 2P - K}{S} \right\rfloor$



Feature Progression in a CNN

VGG16 Architecture



- When classifying images, CNNs trade spatial resolution for feature channels.
- Progressing through the network, note how
 - Height and width decreases, channel count increases
 - Receptive field increases
 - Representations progressively increase in complexity and become more task specific.

Classification Head

- The classification head turns features into a prediction by selecting the class with the largest logit, or class score.

$$\underline{\mathbf{z}} = g_{\theta_c}(\mathbf{h}) \quad \rightarrow \quad \mathbf{z} = \underline{\mathbf{W}}^{(L)} \underline{\mathbf{a}}^{(L-1)} + \underline{\mathbf{b}}^{(L)} \quad \hat{y} = \underset{k}{\operatorname{argmax}} z_k$$

$\mathbf{z}_k \in \mathbb{R}$ is the logit score for class k , $\mathbf{z} \in \mathbb{R}^K$ is the logit vector

penultimate output of MLP $\mathbf{a}^{(L-1)} \in \mathbb{R}^{d_{L-1}}$

Last layer weight matrix $\mathbf{W} \in \mathbb{R}^{K \times d_{L-1}}$ and offset $\mathbf{b}^{(L)} \in \mathbb{R}^K$

- Logits are unrestricted real-valued scores. Use the soft-max operation to convert it into a probability distribution over the K classes.

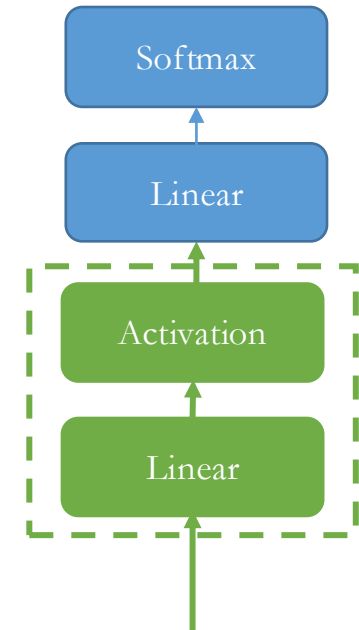
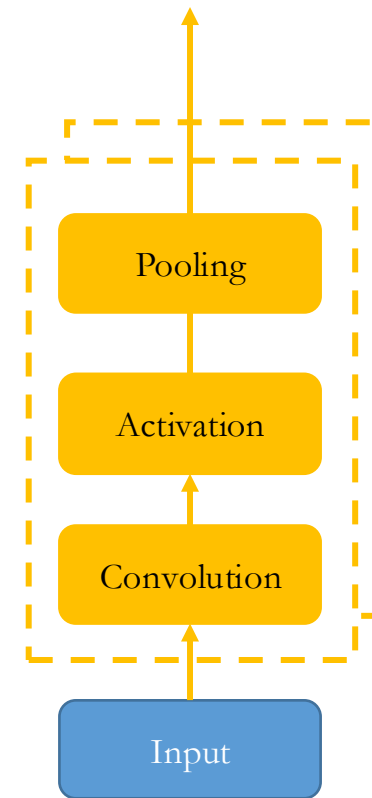
$$p_k = \frac{\exp(z_k)}{\sum_{j=1}^K \exp(z_j)} \quad \mathbf{p} \in [0,1]^K, \quad \sum p_k = 1$$

Softmax is useful because it: produces nonnegative values that sum to one. It allows p_k to be interpreted as the model's predicted probability for class k and provides the probabilities used by the standard multiclass cross-entropy loss.



General Architectures

- ConvBlock (module)
 - Convolution
 - Activation
 - Pooling
- Classification
 - Linear + Activation + Softmax
- Object detection
- Image segmentation
- ...



How many blocks?
What is the configuration
of each convolution layer?

A CNN Backbone Supports Many Tasks

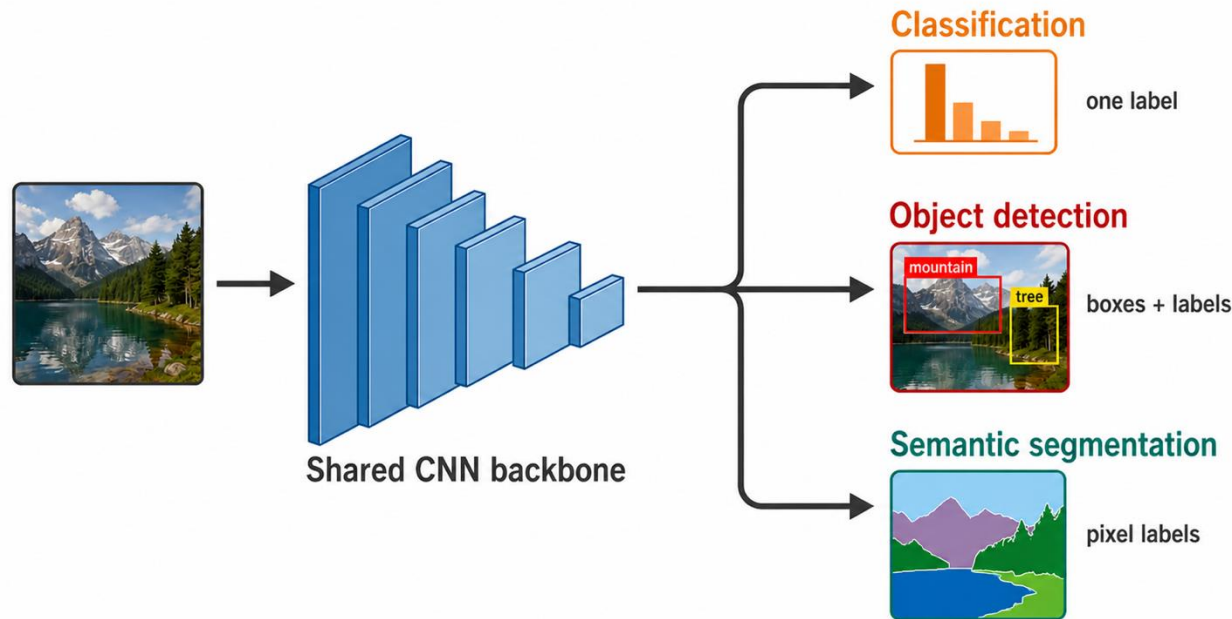


Image Classification: Assigning a discrete class label to entire image

Object Detection: classifying objects & localize with a bounding box

Semantic Segmentation: Associate each pixel with a class label

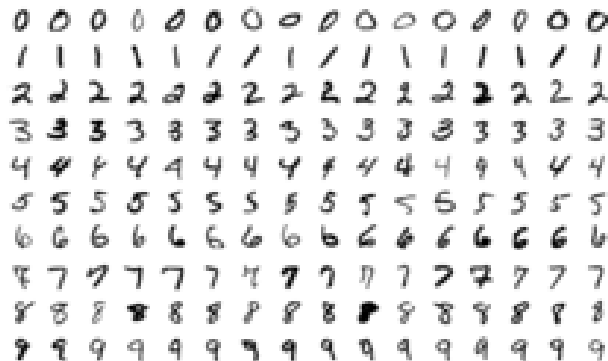
Instance Segmentation: Associate each pixel in the image with the object to which it belongs. (not shown; segment the individual trees)

- The CNN backbone produces reusable spatial feature maps.
- A task-specific **head** converts those features into the required output.
- The same pretrained backbone can be adapted to multiple vision tasks.

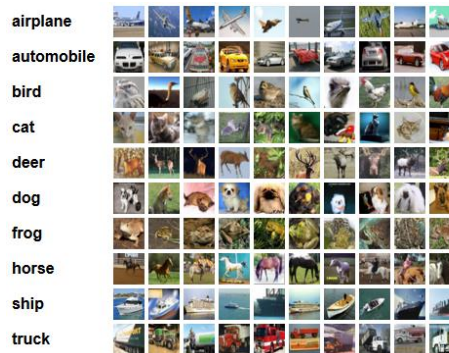
Where Does the Data Come From?

- A supervised dataset is a collection of labeled examples. The dataset's **scale, diversity, labels, and biases** determine what the model can learn and how well it generalizes.
- ImageNet's scale and standardized benchmark enabled the training and comparison of large CNNs, accelerating architectural progress after AlexNet's breakthrough in 2012.

Dataset	Year	# classes	# images	Resolution
MNIST	1998	10	70, 000	28x28
CIFAR10	2009	10	60, 000	32x32
ImageNet	2009	1000 **	1.2 million **	variable



https://en.wikipedia.org/wiki/MNIST_database

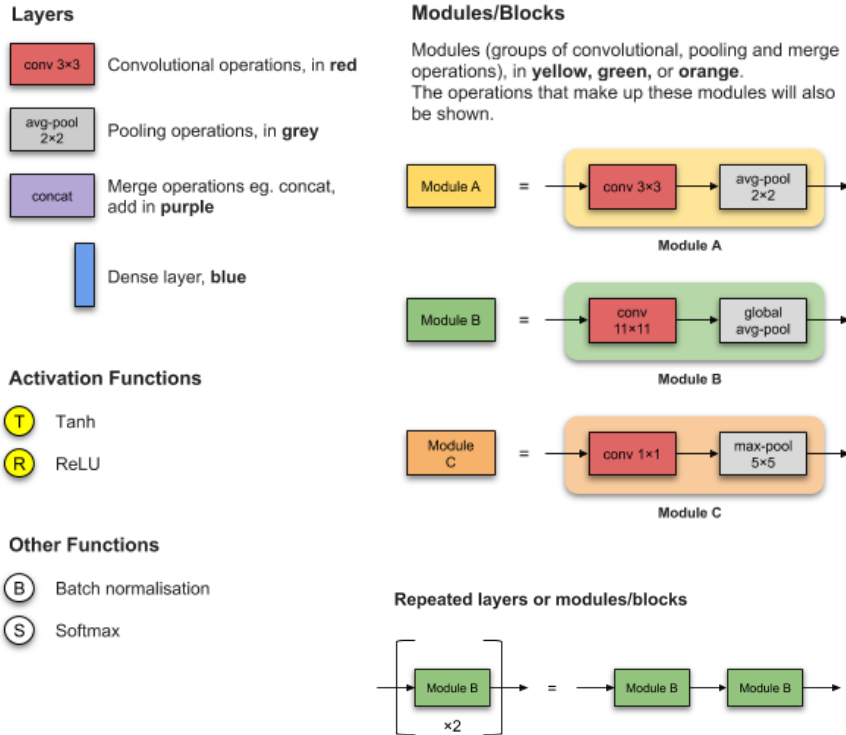


<https://www.cs.toronto.edu/~kriz/cifar.html>

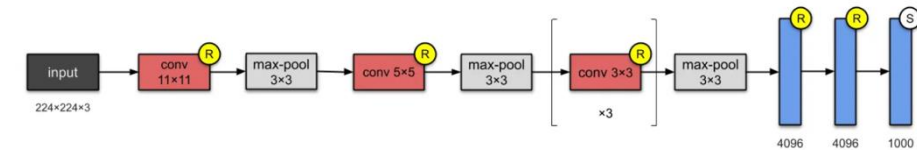


<https://medium.com/syncedreview/sensetime-trains-imagenet-alexnet-in-record-1-5-minutes-c944ab049b2c>

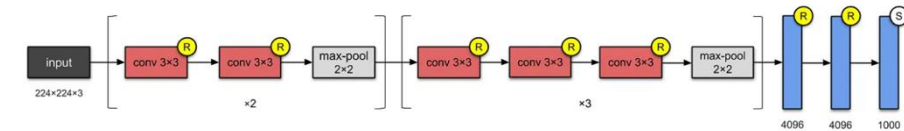
Visualization of ConvNet Architectures



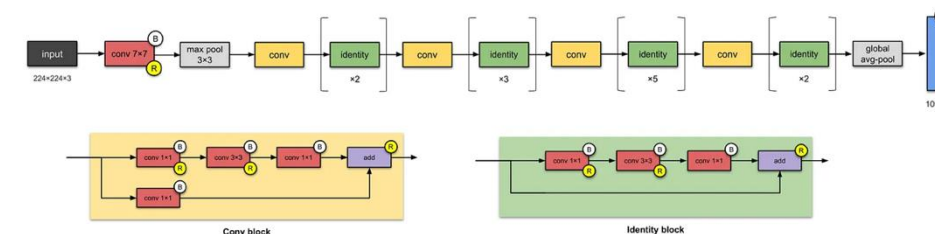
AlexNet



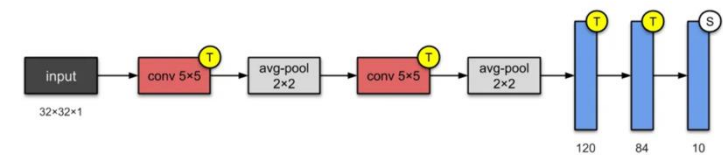
VGG-16



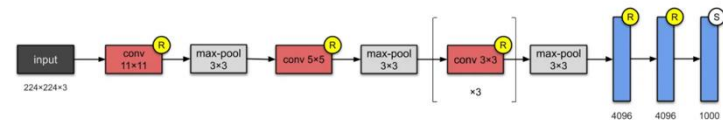
ResNet-50



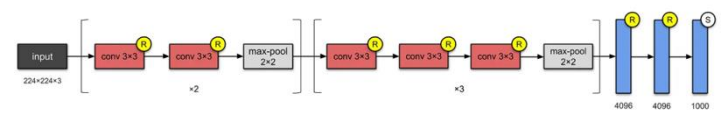
LeNet-5



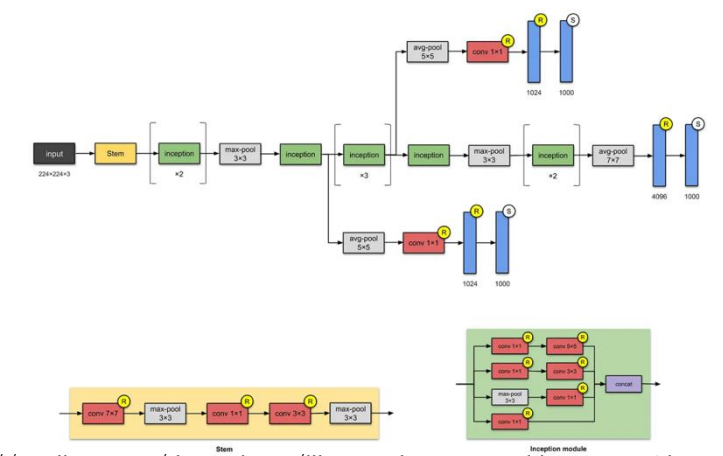
AlexNet



VGG-16



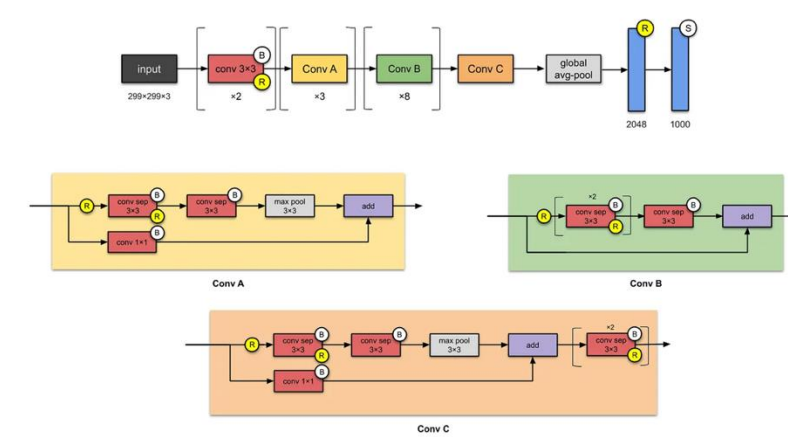
Inception-v1



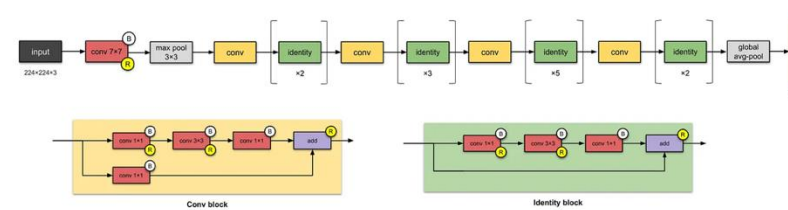
Modern CNNs are built from repeated blocks with recurring design patterns:

- Stages of repeated modules progressively transform the features.
- Between stages, spatial resolution decreases, # of channels increases.
- Later architectures had parallel paths, bottlenecks, skip connections to combine and reuse features.
- Final features are aggregated and passed to a task-specific head.

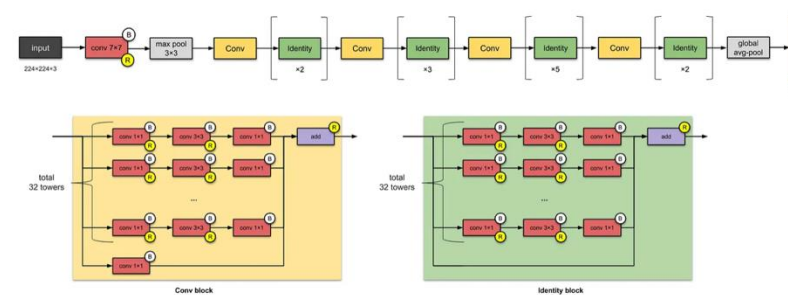
Xception



ResNet-50

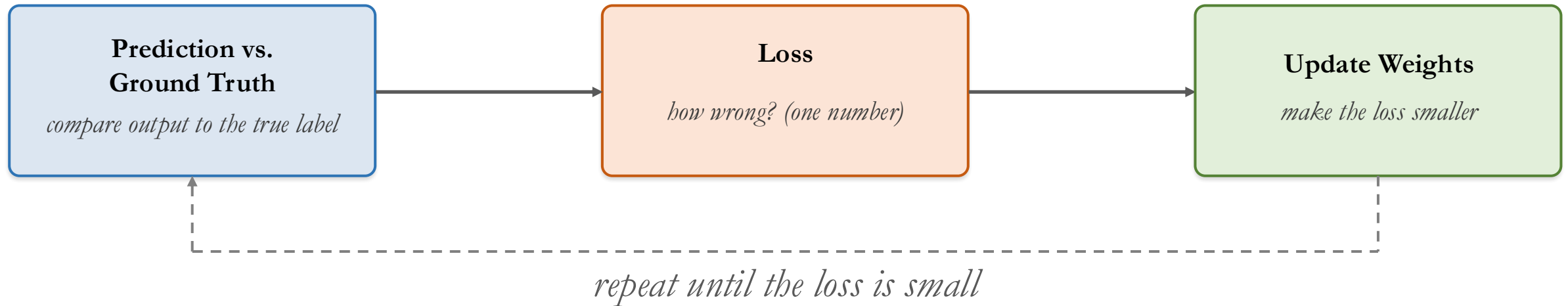


ResNeXt-50



Training a Network: The Loss Function

A loss function produces a single number measuring how wrong a prediction is for the training data.



Training a Network: The Loss Function

1. Predicting a location (x,y)

$$L_{\text{loc}} = (\hat{x} - x)^2 + (\hat{y} - y)^2$$

Distance-based — naturally continuous, so “a little better” is meaningful.

2. Naive idea for classification: right or wrong?

$$L_{0/1}(\hat{y}, y) = \begin{cases} 0, & \hat{y} = y \text{ (correct)} \\ 1, & \hat{y} \neq y \text{ (wrong)} \end{cases}$$

Only two possible values — no notion of “almost right,” so it gives no useful learning signal.

we need something smoother ↓

3. Cross-entropy loss (used in practice)

$$L_{\text{CE}} = -\log(p_y)$$

Continuous — softly penalizes low confidence in the correct class.

The Bigger Picture on CNNs

Core Takeaways

- Convolution applies learned local filters and combines local connectivity w/ shared parameters.
- Multiple filters produce feature channels; stacking layers increases receptive fields and builds progressively richer representations.
- Specific architectures are not important; instead, remember that CNNs are typically organized into blocks and stages, with spatial resolution decreasing and channel count increasing.
- A CNN backbone extracts features; a task-specific head converts them into logits and predictions.

Topics Beyond Our Course

- **Training and generalization:** backpropagation, optimization, normalization, regularization, and data augmentation. (How are CNNs trained?)
- **Modern representation learning:** pretraining, transfer learning, self-supervised learning, and vision transformers. (Beyond supervised learning.)
- **Practical deployment:** computational efficiency, robustness, interpretability, uncertainty, and dataset bias, etc. (Running CNNs in the real world)

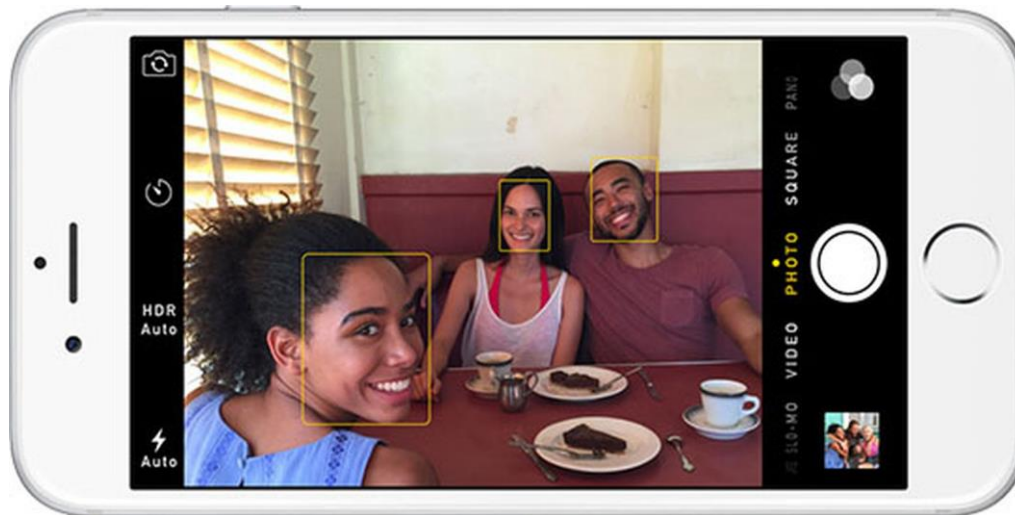
Object Detection

Task definition, Window-Based Detection Paradigm, Evaluation

Object Detection Applications

Applications

- Face detection: camera focus
- Surveillance: cars, people, animals
- Indexing: get objects from images for search



<https://www.macrumors.com/2017/02/16/iphone-3d-laser-scanner-facial-recognition/>

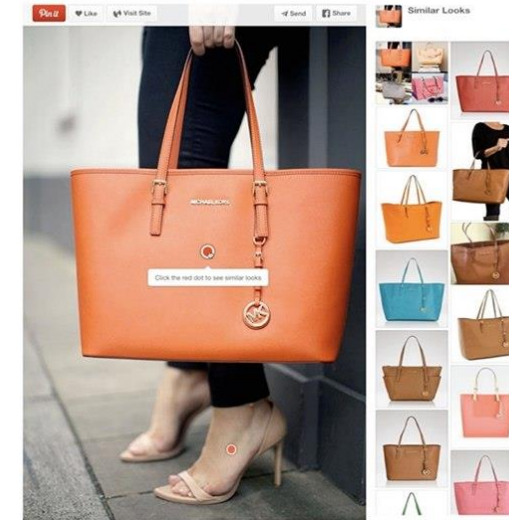
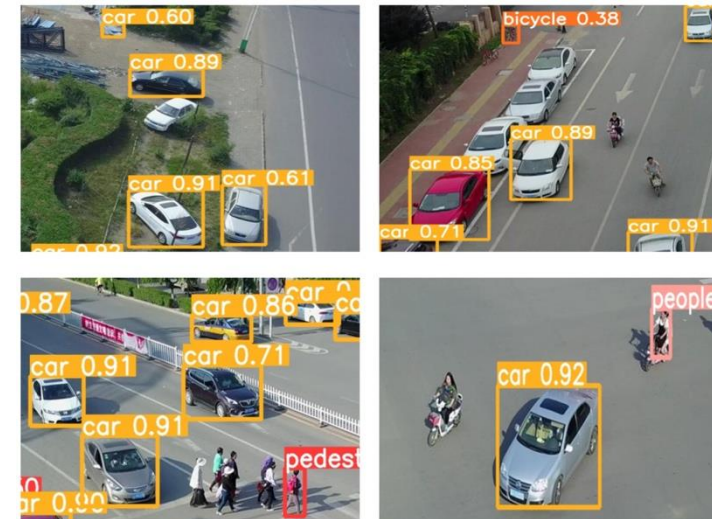


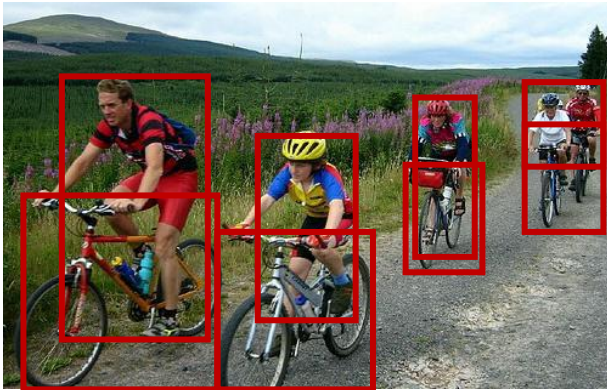
Figure 1: Similar Looks: We apply object detection to localize products such as bags and shoes. In this prototype, users click on automatically tagged objects to view similar-looking products.



<https://www.mdpi.com/2079-9292/12/15/3296>

Object Detection Task

- Detect the location of all object instances of all classes
- Training labels:
a list of <class, bounding box of each object instance>
- Prediction outputs:
a list of <class, probability, bounding box of each object instance>



Training label:

bicycle (10, 100, 110, 110) (200, 200, 180, 80) ...
People (200, 80, 71, 71) (300, 50, 20, 80) ...

Prediction output:

bicycle 0.9 (9, 93, 100, 111), 0.8 (200, 200, 180, 80), ...
people 0.8 (200, 80, 71, 71), ...
....

“What is where?”

For object i , we predict

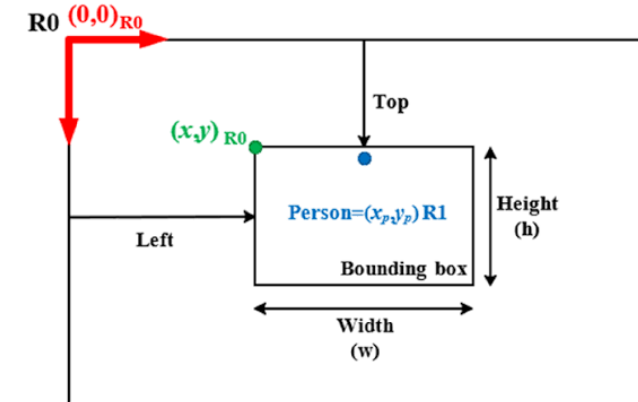
$(\underline{c_i}, \underline{s_i}, \underline{x_i}, \underline{y_i}, \underline{w_i}, \underline{h_i})$

c_i : class/category

s_i : confidence score

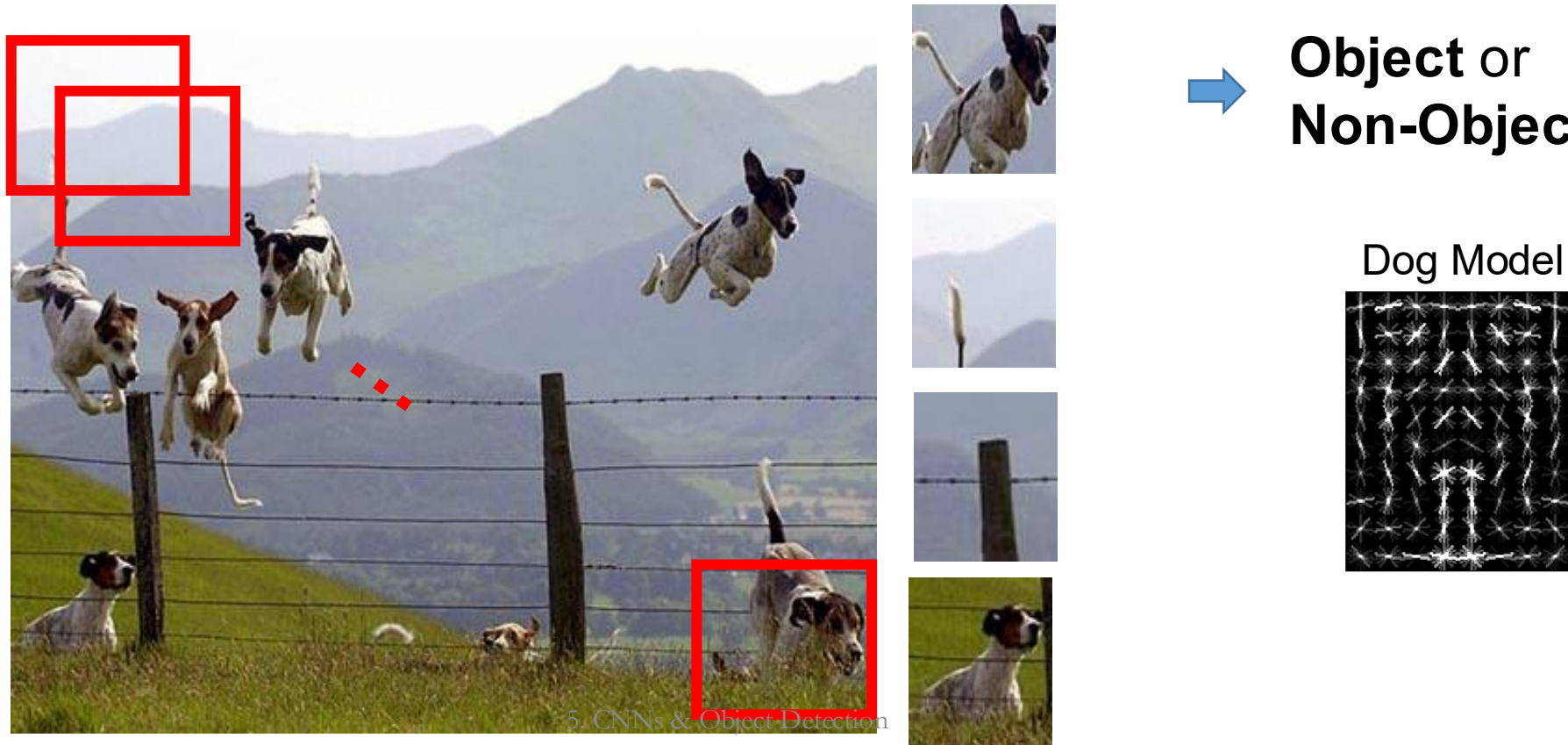
x_i, y_i : coordinate of the top-left corner of the box

w_i, h_i : width and height of the box



Window-Based Object Detection

- Build templates to differentiate object patches from background patches



Challenges in Modelling the Object Class



Illumination



Object pose



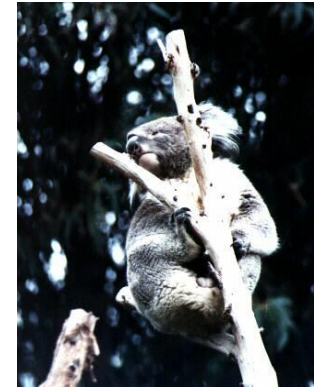
‘Clutter’



Occlusions



Intra-class appearance



Viewpoint

Challenges in Modelling the Non-Object Class

True
Detections



Bad
Localization



Confused with
Similar Object



Misc. Background



Confused with
Dissimilar Objects



Window-Based Object Detection Framework

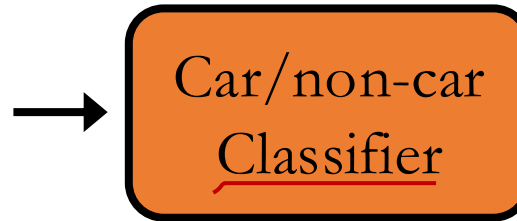
-
- The diagram illustrates a window-based object detection framework, divided into two main phases: training and testing. The training phase (step 1) involves building or training an object model, which includes choosing a representation and learning or fitting parameters of the model or classifier. The testing phase (steps 2-4) involves generating candidate windows in a new image, scoring the candidates, and resolving detections, which includes rescore candidates based on the entire set.
- training*
1. Build/train object model
 - Choose a representation
 - Learn or fit parameters of model / classifier
- testing*
2. Generate candidate windows in new image
 - test all locations and scales
 3. Score the candidates
 4. Resolve detections
 - Rescore candidates based on entire set

Window-Based Object Detection Framework

training

1. Build/train object model
 - Choose a representation
 - Learn or fit parameters of model / classifier

Given a feature representation,
train binary classifier



Yes, car.
No, not a car.

Former (non-deep) object models replaced by neural networks. Advantage is that it integrates the feature learning together with the object model.

Window-Based Object Detection Framework

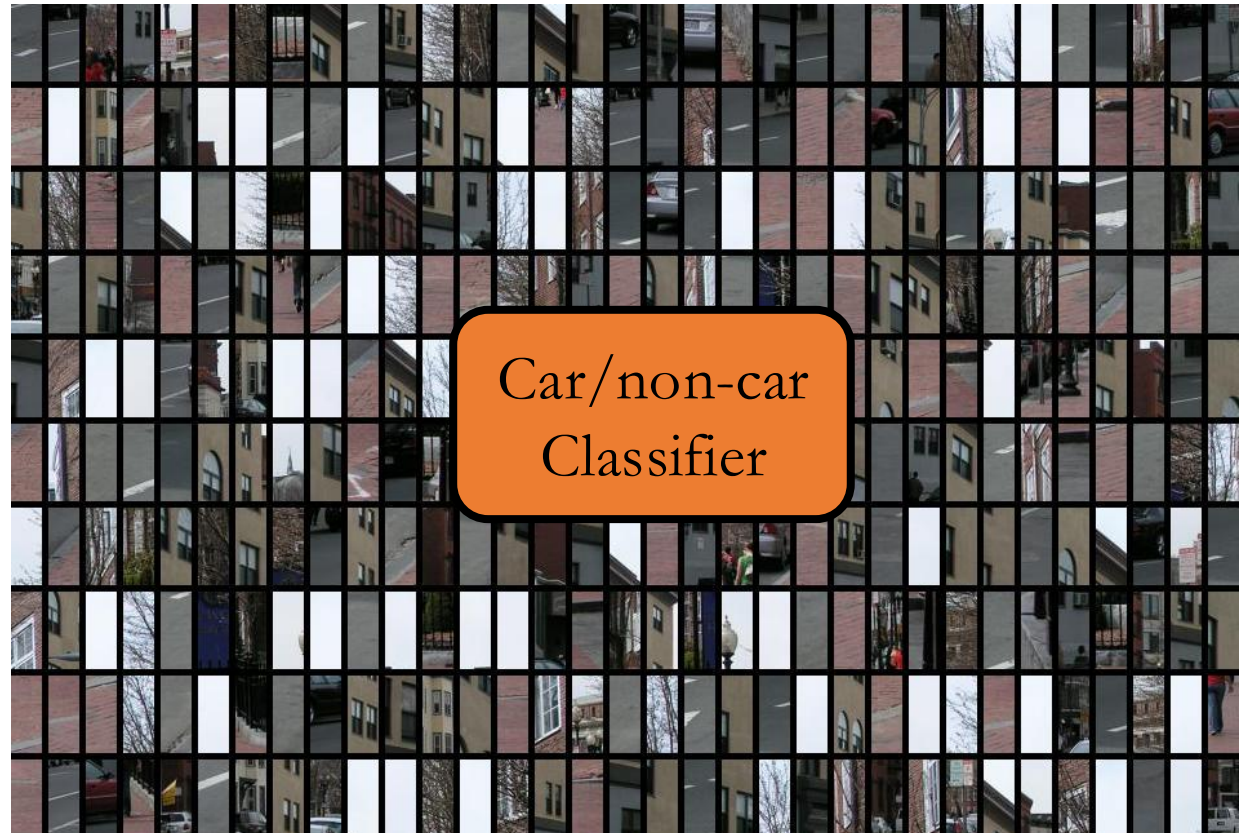
2. Generate candidate windows in new image
 - test all locations and scales



Note – Template did not change size

Window-Based Object Detection Framework

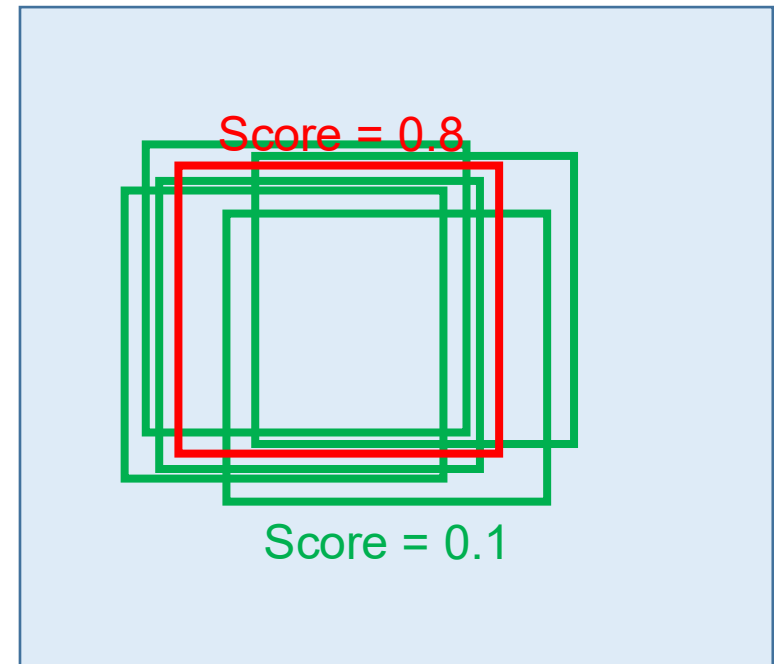
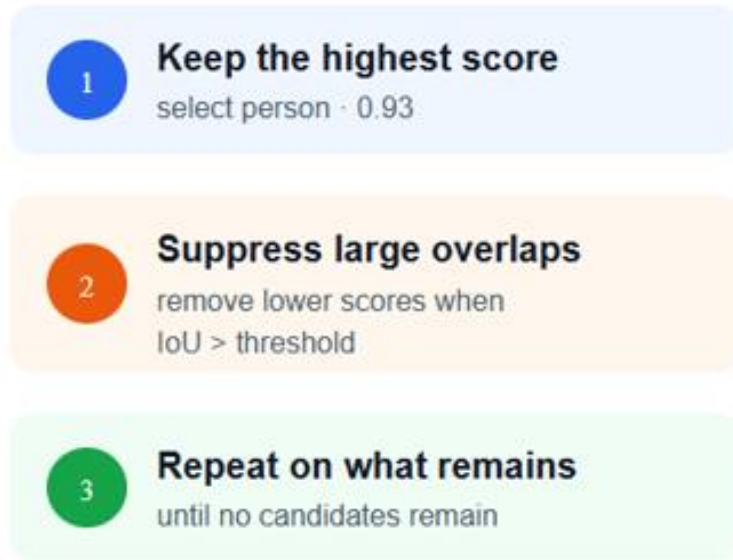
3. Score the candidates



Each window is classified separately.

Window-Based Object Detection Framework

Threshold, non-maximum suppression (NMS)

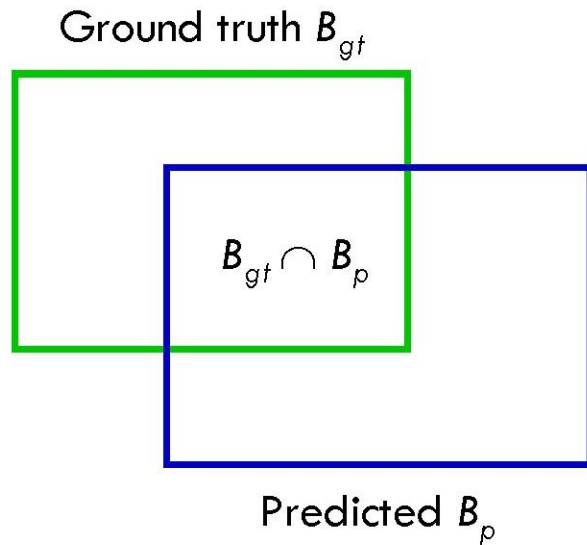


4. Resolve detections
 - Rescore candidates based on entire set

How to Evaluate Performance?

- Area of Overlap (AO) Measure

*sometimes also called
intersection over union (IoU)*



$$AO(B_{gt}, B_p) = \frac{|B_{gt} \cap B_p|}{|B_{gt} \cup B_p|}$$

- Need to define a threshold t such that $AO(B_{gt}, B_p)$ implies a correct detection: 50%

Detection Errors

After detection score thresholding & NMS:

- **True Positive (TP)** – detected objects aligned with the ground truth by IoU threshold t (usu. $t=0.5$)
- **False Positive (FP)** – detected objects w/out corresponding ground truth.
 - detections which do not meet IoU threshold are counted as false positives!
- False Negative (FN) – objects marked in ground truth which were not detected
- True Negative (TN) – no meaning for object detection.

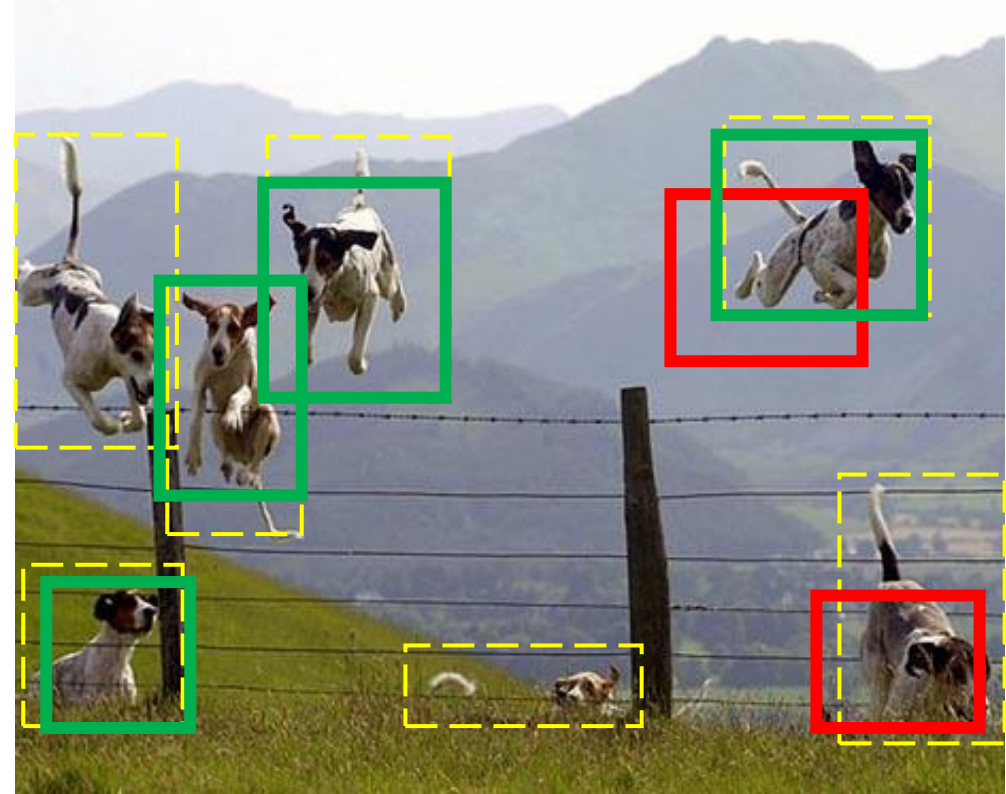
$$\text{Precision} = \frac{TP}{TP + FP}$$

Of all predicted objects, how many were correct?

$$\text{Recall} = \frac{TP}{TP + FN}$$

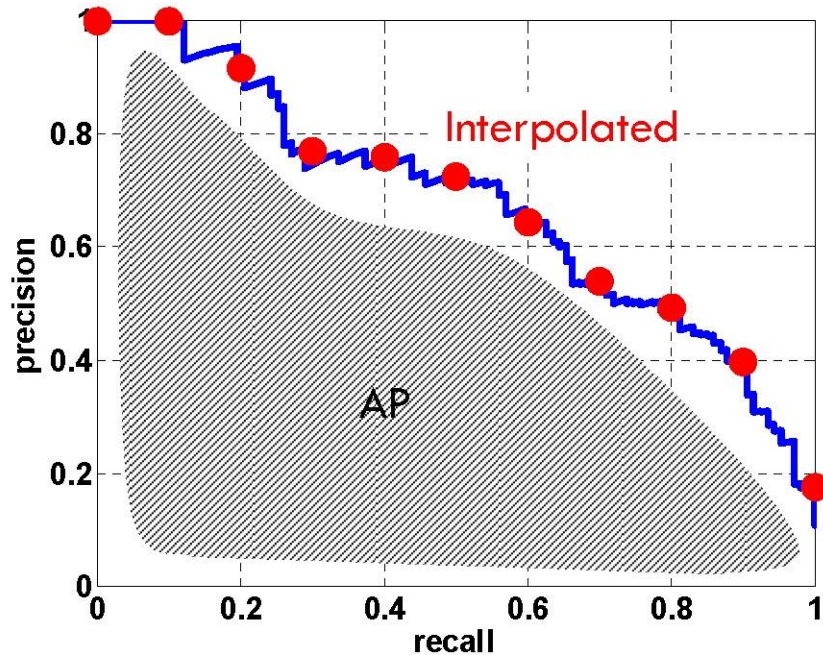
Of all real objects, how many were found?

Dashed lines are ground truth
solid lines are detector outputs
4TP, 3FN, 2FP



How to Evaluate?

TL;DR mAP is a number from either 0-1 or 0-100%; high is good



A good AP requires both high recall & high precision

- Low precision: many false positives
- Low recall: many false negatives
- AP penalizes methods with high precision but low recall (well-localized bounding boxes for only a few objects, misses many others).

- Precision-recall gives only a single indication of performance (and for only a fixed IOU threshold).
- Average precision $\rightarrow$ more indicative to consider precision values associated over a range of recalls (via averaging)
- For one class c at a fixed IOU threshold τ :
 - **Match and rank detections:** Sort by confidence. Mark each prediction TP or FP using $IoU \geq \tau$ and one-to-one matching.
 - **Sweep the confidence threshold:** Every threshold gives a precision-recall point; the sweep traces the curve.

Summarize, then average

$AP(c, \tau)$ = area under the precision-recall curve

$mAP(\tau)$ = mean of AP across classes

Some protocols also the average over $\tau = .50, .55, \dots, .95$.

CNN-Based Object Detection

R-CNN + faster variants, Two vs. One-Staged Detection, YOLO

From Window-Based to Deep Object Detector

training

1. Build/train object model
 - Choose a representation *a deep*
 - Learn or fit parameters of *model* / classifier

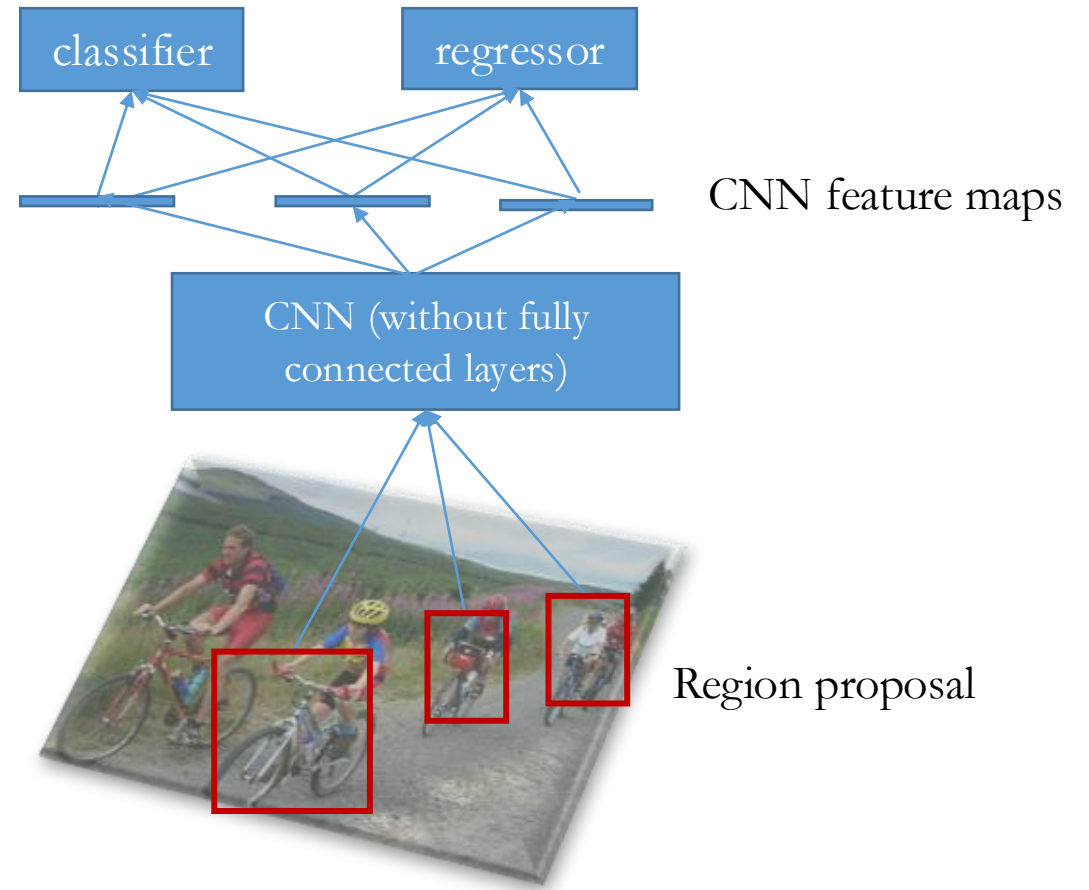
testing

2. Generate candidate windows in new image
 - test all locations and scales
 3. Score the candidates
 4. Resolve detections
 - Rescore candidates based on entire set
- Narrow down to a subset vs. "exhaustive"
 - Extract CNN feature from this region
 - Refine the region boundary w/ a regressor to generate 4 values of bounding box (x,y,h,w).
 - Predict the class label using a classifier

R-CNN: Region-Based ConvNets [[pdf](#)]

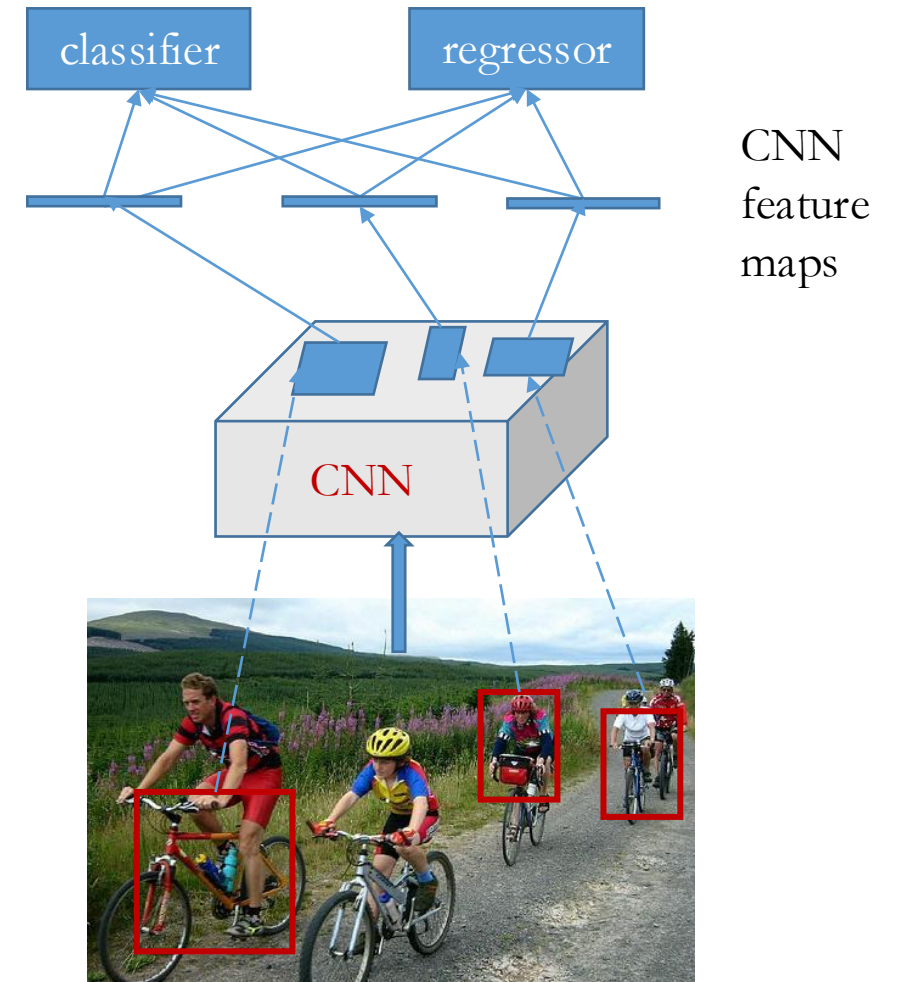
- Generate region proposal (candidate objects)
- Extract CNN features for each region
 - Used popular CNNs: VGG, ResNet, etc.
 - 🐢 v. slow! ~2000 regions, each passes through a CNN
- Regress bounding box (x,y,h,w) for each region
 - Linear regression (L2 loss) for each value
 - Refines regions into final bounding boxes
- Predict label (object class) for each region
 - Linear (SVM) classifier

🐢 ad-hoc training: CNN + label classifier + bounding box regressor



Fast R-CNN & Faster R-CNN

- ~~Remove external region proposal generation (Faster R-CNN)~~
 - ~~Generate region proposal (candidate objects)~~
- Extract CNN features for entire image in one pass
 - Region proposal integrated into neural network
 - Locate region coordinates in CNN feature maps
 - Extract features from regions w/ ROI pooling
- Regress bounding box (x,y,h,w) for each region
 - Linear regression (L2 loss) for each value
 - Refines regions into final bounding boxes
- Predict label (object class) for each region
 - Linear (SVM) classifier

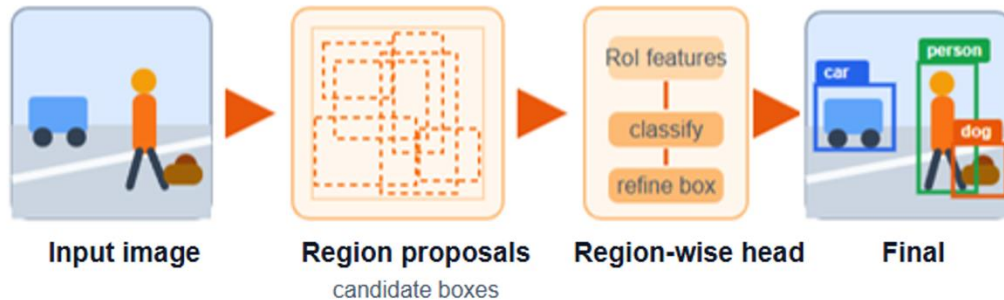


From Two-Stage to One-Stage Detection

TWO STAGE

Propose, then classify and refine

Candidate regions are selected before the final prediction head.



- 1 Stage 1: propose likely object regions
- 2 Stage 2: classify and refine each proposal

Examples: Faster R-CNN, Mask R-CNN

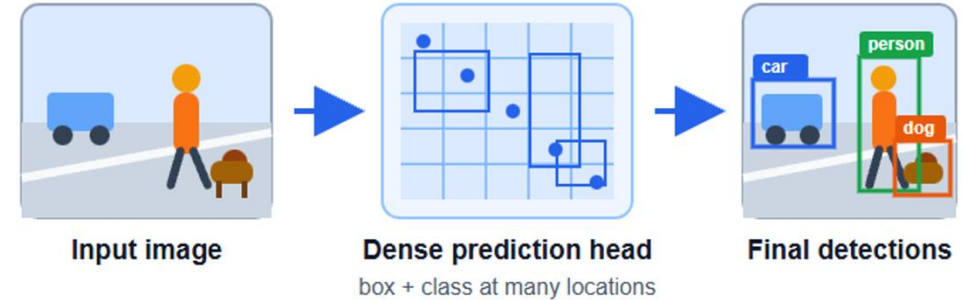
SELECTIVE

Explicit candidate-selection stage

ONE STAGE

Dense, direct prediction

Boxes and classes are predicted across the image in one pass.



- 1 A single, end-to-end prediction pipeline

No separate proposal stage: all locations compete to produce final boxes.

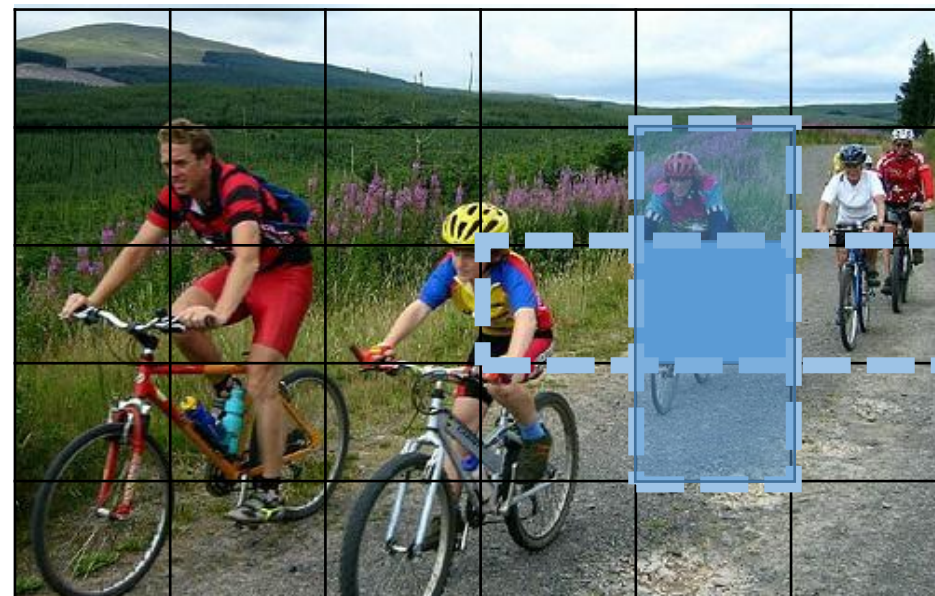
Examples: YOLO, SSD, RetinaNet

DIRECT

Typically simple and fast at inference

YOLO (You Only Look Once) Object Detector

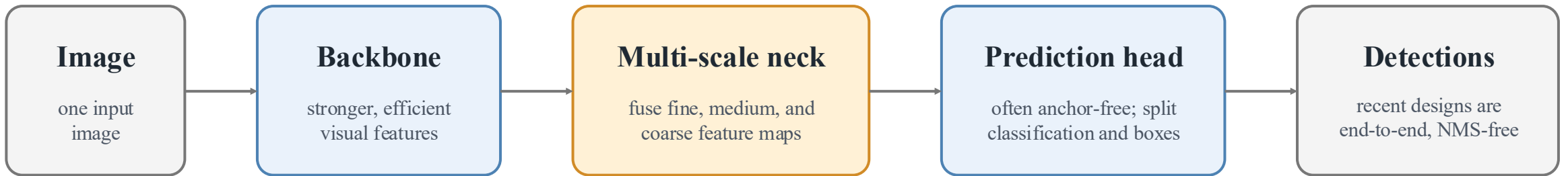
- Partition the image into array of cells
- Each cell associated w/ predefined anchors (v2)
 - Bounding boxes of predefined sizes & aspect ratios
- Make predictions from each grid location and each aspect ratio
 - Objectness: P(the predicted box has object)
 - If no object skip, otherwise predict
 - Class score: P(category | object)
 - Anchor box adjustment: P(coordinates | object)



$$\mathcal{L} = \mathcal{L}_{\text{class}} + \lambda_{\text{box}} \mathcal{L}_{\text{box}} + \lambda_{\text{obj}} \mathcal{L}_{\text{objectness}}$$

Modern YOLOs, from 2016 to 2026

- Retains core, one-stage prediction of object classes and locations, with speed as a central design goal



- **Multi-scale features:** feature pyramids combine fine spatial detail with deeper semantic features, improving detection across object sizes.
- **Decoupled prediction:** predict classes and boxes w/ separate branches, no predefined anchor boxes
- **Accuracy & Speed:** Better training methods and more efficient network components.
- **Simpler, broader outputs:** end-to-end w/out NMS; broader ecosystem supports segmentation, pose estimation, tracking, and open-vocabulary recognition, etc.

Summary

Learning Representations

- Image classification maps an image to class scores and then selects a predicted label.
- Traditional vision fixes the feature extractor;
- Deep learning learns the representation and classifier jointly.

Convolutional Features

- Local connectivity and shared kernels exploit image structure while greatly reducing parameter count.
- Multiple filters create channels; stacking layers expands receptive fields and builds hierarchical representations.

CNN Architectures

- CNNs organize convolution, nonlinearity, and down-sampling into repeated blocks and stages.
- A backbone extracts features; a task-specific head converts them into logits, boxes, masks, etc.

Object Detection

- Window-based detection applies an image classifier at many locations and scales
- Deep detectors share CNN feature maps and predict classes and bounding boxes from them
- Two-stage models refine region proposals, while one-stage models predict densely and directly.

Extra Readings

- [Torralba](#) 12.2-12.5 Perceptrons, MLPs, deep networks
24.1-7, 24.9-11 CNNs, sliding applications, architectures
50.3-4 Image classification, object detection/localization
- [Szeliski](#) 5.3 Deep Neural Networks
5.4 Convolutional Neural Networks
- [Klette](#) 10 Object Detection

FAQs

Pre-Amble

Q: Does high test accuracy mean that an image classifier is reliable?

A: Not necessarily. Accuracy can conceal class imbalance, systematic failure on particular groups, dataset bias, and poor performance on images that differ from the test distribution.

Q: Why must the training, validation, and test sets be separate?

A: Training fits the parameters, validation guides design choices, and testing estimates generalization performance on unseen data. Repeatedly consulting the test set indirectly turns it into training data.

Q: Are logits probabilities?

A: No. Logits are unrestricted class scores. Softmax converts them into nonnegative values that sum to one, although these values are not automatically well calibrated.

Q: If softmax produces probabilities, why can we use the logits for prediction?

A: Softmax preserves the ordering of the scores, so $\arg \max_k z_k = \arg \max_k \text{softmax}(z)_k$

Q: Where exactly is the boundary between the feature extractor ϕ and classifier g ?

A: It is a modelling convention. We usually call the layers producing the reusable representation h the feature extractor and the final task-specific mapping the classifier.

Q: Was nothing learned in traditional computer vision?

A: The classifier g was often learned. What was usually hand-designed was the feature extractor ϕ , using features such as edges, HOG, SIFT, or filter-bank responses.

Q: Why is jointly learning the feature extractor ϕ and classifier g useful?

A: It allows the representation to adapt to the task rather than forcing the classifier to work with features chosen in advance.

Q: Is end-to-end learning always better?

A: No. It generally needs substantial representative data and computation, and it can learn unwanted correlations. Hand-designed structure can still be valuable when data are limited or constraints are strong.

Q: Can a feature extractor trained for one task be reused for another?

A: Often yes. Transfer learning reuses a pretrained representation and replaces or fine-tunes the task-specific head.

Q: What happens if a new class is introduced after training?

A: A closed-set classifier cannot predict a class that has no output. Its head—and usually at least part of the network—must be retrained or adapted.

Q: Does more training data automatically solve generalization?

A: Only if the added data cover the variation encountered at deployment. A very large but systematically biased dataset can still produce a biased model.

Q: What does the parameter vector θ contain?

A: It collects all learned weights and biases. In the staged notation, it can be partitioned into feature-extractor parameters θ_f and classifier parameters θ_g .

FAQs

Convolution & Feature Maps

Q: Why use convolution instead of a fully connected layer for an image?

A: Convolution uses local connectivity and shares parameters across locations. This exploits spatial structure and requires far fewer parameters.

Q: Are CNN ‘convolutions’ actually convolutions?

A: Most implementations compute cross-correlation—the kernel is not flipped. Because the kernel is learned, this does not reduce the model's representational capacity.

Q: What does one feature channel represent?

A: It is one response map produced by a learned filter or combination of earlier features. A channel need not have one simple human-readable meaning.

Q: Does a convolutional kernel operate on only one input channel?

A: No. Each output filter spans all input channels, sums their contributions, and produces one output channel.

Q: What is the difference between kernel size and receptive field?

A: Kernel size is the local window used in one layer. The receptive field is the region of the original input that can influence a later activation.

Q: Why reduce spatial resolution while increasing channels?

A: Lower resolution reduces computation and increases context; additional channels retain a richer set of features. This is common, but not mandatory.

Q: Are CNNs translation invariant?

A: Convolution is translation equivariant under ideal conditions. Pooling or global aggregation can encourage invariance; stride, padding, and boundaries can break exact equivariance.

Q: Is a fully connected layer an MLP?

A: One fully connected layer is an affine map. An MLP stacks affine maps with nonlinear activations. A classification head may use either.

Q: Does parameter sharing mean that all filters in a layer are identical? 04. Filter Banks, Texture & Learned Features

A: No; the “sharing” refers to the convolution operation, in which the filter is reused spatially .

Q: Why is the operation in CNNs called convolution when it is often cross-correlation?

A: Most libraries do not flip the kernel, so mathematically they compute cross-correlation. Because the kernel itself is learned, the distinction has little practical effect on the model.

Q: Does a convolutional layer have fewer parameters because the image is smaller?

A: Its parameter count depends mainly on kernel size and channel counts, not image height and width. Larger images still require more computation and memory.

Q: Are CNNs translation invariant?

A: Convolution is approximately translation equivariant: shifting the input shifts the feature map. Invariance comes from operations like pooling, global aggregation, etc. but is imperfect in practice.

Q: Why use padding if it introduces artificial values at the image boundary?

A: Padding controls output size and prevents feature maps from shrinking too quickly. The trade-off is that boundary activations are influenced by the chosen padding rule.

Q: Is strided convolution the same as pooling?

A: Both downsample, but a strided convolution learns how to combine values while pooling uses a fixed aggregation such as maximum or average.

Q: Why not use dilation everywhere to obtain a large receptive field?

A: Large dilation can sample the input too sparsely, producing gridding artefacts and missing local detail. It is most useful when large context and dense spatial output are both needed.

Q: Is the theoretical receptive field the region the network actually uses?

A: Not necessarily. Every pixel in the theoretical field can influence the output, but the effective influence is often concentrated near its centre.

Q: Why do CNNs usually reduce spatial resolution while increasing channel count?

A: Lower resolution reduces computation and allows each activation to summarize a larger image region, while additional channels preserve different kinds of information about that region.

Q: If vision transformers are modern, why should we still study CNNs?

A: CNNs remain widely used and introduce enduring ideas—locality, parameter sharing, multiscale processing, feature hierarchies, backbones, and task-specific heads—that also help explain modern architectures.

FAQs

Object Detectors & Evaluation

Q: Why can't we simply run an image classifier once to perform detection?

A: A classifier says what is present globally but not how many instances exist or where they are. A detector must produce a variable-size set of localized predictions.

Q: How does a window classifier become an object detector?

A: Apply it at many locations and scales, retain high-scoring windows, and consolidate overlapping predictions. This historical pipeline motivates many components of modern detectors.

Q: Why must window-based detectors search multiple scales?

A: The classifier expects an object at roughly a fixed size, while objects can occupy very different fractions of the image. Image pyramids or differently sized windows provide scale coverage.

Q: Why is modelling the background class unusually difficult?

A: An object class has bounded variation, but “background” includes almost everything else. Hard-negative mining is therefore often needed to find convincing false positives.

Q: Is a detector's confidence score the probability that its prediction is correct?

A: Not necessarily. It is a ranking score derived from objectness and class evidence; it may require calibration before it can be interpreted as a reliable probability.

Q: What is the difference between class confidence and objectness?

A: Objectness asks whether a proposed region contains any object. Class confidence asks which category the object belongs to; implementations may combine these scores.

Q: Why is non-maximum suppression needed?

A: Nearby candidate boxes often describe the same object. NMS retains a strong prediction and suppresses sufficiently overlapping alternatives.

Q: Can NMS remove correct detections?

A: Yes. In crowded scenes, boxes for different nearby objects may overlap strongly, causing ordinary NMS to suppress a valid prediction.

Q: What happens when several predictions overlap the same ground-truth object?

A: Under standard evaluation, only one can be matched as a true positive. Additional detections of that object are counted as false positives.

Q: Why is there no useful count of true negatives in object detection?

A: The image contains an enormous—and not uniquely defined—number of possible background boxes. Consequently, precision and recall are more meaningful than accuracy.

Q: What is IoU used for?

A: IoU measures box overlap. It is used to match predictions to ground truth and to decide which overlapping predictions NMS should suppress.

Q: Is mAP a single universally comparable metric?

A: No. Its definition depends on the benchmark, IoU thresholds, class averaging, and sometimes object sizes. Results are comparable only under the same evaluation protocol.

Q: What changed from R-CNN to Fast R-CNN and Faster R-CNN?

A: R-CNN ran the CNN separately for every proposal. Fast R-CNN shared one image-level feature map but still used external proposals. Faster R-CNN learned the proposals with a region proposal network.

Q: Are two-stage detectors always more accurate and one-stage detectors always faster?

A: That is the historical tendency, not a rule. Modern design, training, input resolution, and deployment hardware can be more important than the one-stage/two-stage label.

Q: What remains fundamentally the same across modern YOLO versions?

A: They retain dense, one-stage prediction with speed as a central goal. Major changes concern backbones, multiscale feature fusion, anchor-free prediction, separated heads, training objectives, and increasingly end-to-end post-processing.